EK2370 Build Your Own Radar System
Project Course Final Report

Group 5
Venkata Lakshmi Roshini Suram, Annalisa Coppi, Mengwei Wu, Carl Clauson

May 4, 2026

1 Introduction

Radar systems operate by transmitting electromagnetic waves and analyzing the reflected signals to determine the range and velocity of objects. Short-range radars at 2.4 GHz and 5.8 GHz are widely used in applications such as motion detection and biomedical monitoring. This project investigates short-range radar systems operating at 2.4 GHz and 5.8 GHz using both analog and software-defined approaches. In the first stage, a commercial off-the-shelf (COTS) radar baseband circuit was built. This circuit supports continuous-wave (CW) and frequency-modulated continuous-wave (FMCW) modes for range and velocity measurements. Baseband signals were digitized through a USB sound card and processed in MATLAB. In the second stage, a 5.8 GHz software-defined radar (SDR) was implemented. This platform allows flexible waveform generation and digital signal processing. Together, the two stages provide hands-on experience in building radar system, illustrating the complete radar signal chain from waveform generation to data analysis.

2 COTS Radar at 2.4 GHz

The 2.4 GHz COTS radar system was built using discrete analog components. The setup consists of a transmit chain, a receive chain, and a baseband circuit for intermediate-frequency (IF) processing. After completing all circuit connections and verification, measurements were performed in CW and FMCW modes to analyze the velocity-time, range-time, and range-velocity relationships, followed by SAR imaging for spatial resolution assessment.

2.1 Hardware Development

2.1.1 Baseband circuit design

The baseband circuit includes three modules, shown in Fig. 1:

- The LM2940CT Low-dropout regulator converts the 12 V DC supply from the USB-to-DC cable into a stable 5 V reference voltage used to supply the XR2206 modulator and NE5532 video amplifier.
- The XR2206 Modulator generates a triangle waveform to tune the VCO and a synchronized square sync signal.
- The NE5532 Video amplifier processes the IF output from the mixer and provides signal amplification and low-pass filtering.

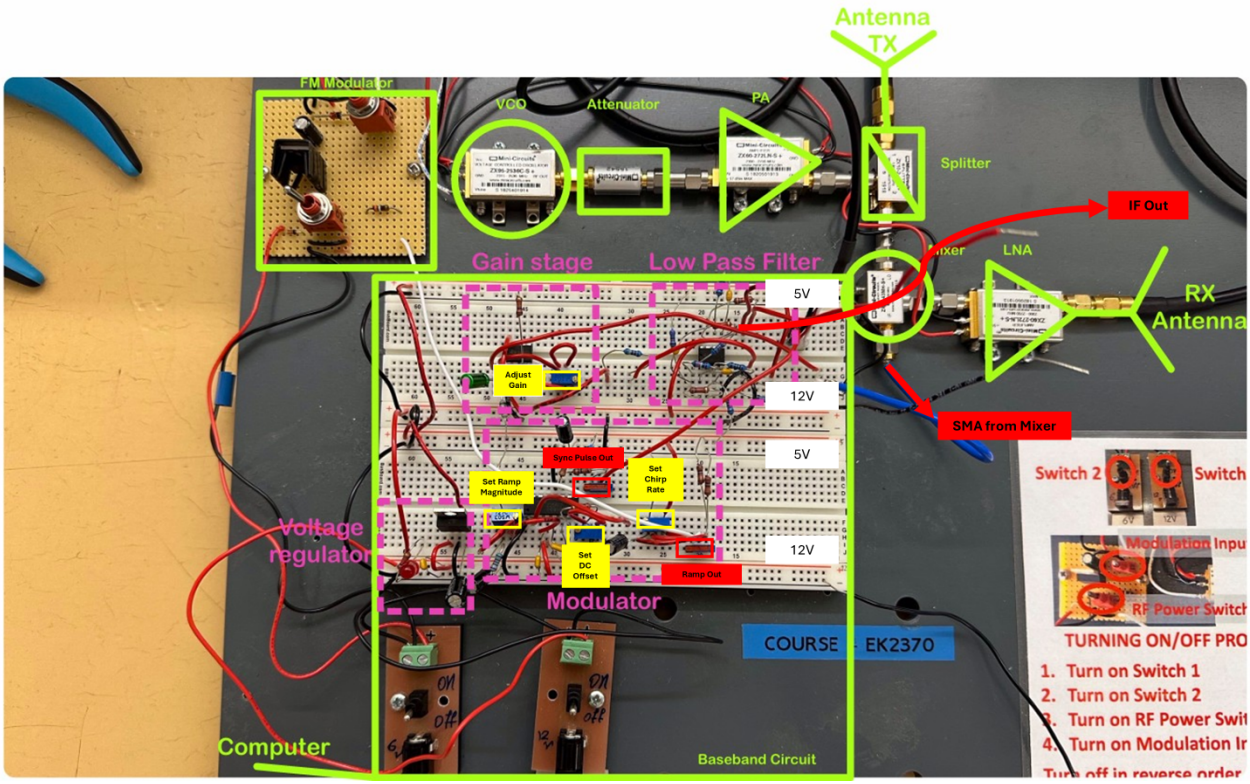


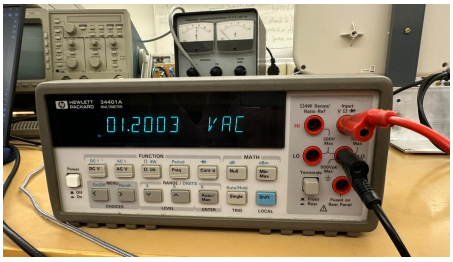
Figure 1: Baseband circuit

Each stage is built and verified separately on a breadboard before final integration.

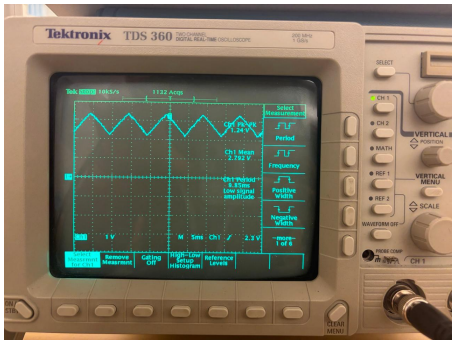
2.1.2 Waveform verification

The XR2206 Modulator outputs were verified using an oscilloscope.

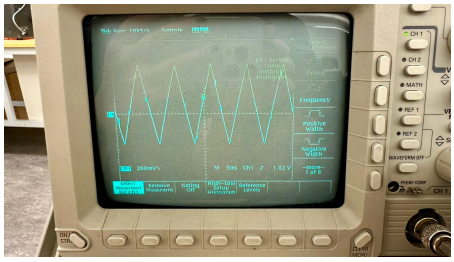
- Pin 2: generates a triangular wave with **1.2 V_{pp}** ramp amplitude and **2.8 V** DC offset (adjustable 2.2–3.4 V) that ensures the VCO Vtune sweeps correctly between 2.4–2.5 GHz. The total chirp period is **10 ms** (5 ms up + 5 ms down). The verification is shown in Fig. 2.
- Pin 11: generates a square wave synchronized with the start of the up-chirp that provides the timing reference for FMCW processing. The verification is shown in Fig. 3.
- Toggle switches: select between CW mode and FMCW mode.



(a) Ramp amplitude



(b) DC offset



(c) Chirp period

Figure 2: Triangular waveform verification

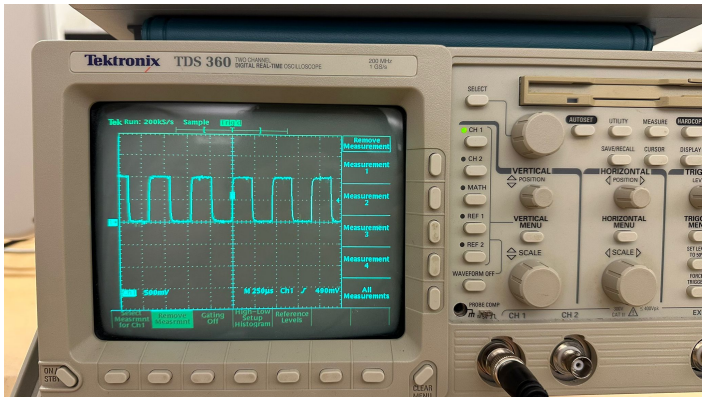


Figure 3: Square waveform verification

2.1.3 3 dB Roll-off of IF filter

To evaluate the low-pass IF filter performance, a sinusoidal input signal from 1 kHz to 15 kHz was applied, and the output amplitude was recorded using both a multimeter and an oscilloscope. The measurement probe was connected **after the 47 kΩ series resistor**, corresponding to the actual observation point used in the radar system.
The –3 dB point, which defines the filter cutoff, corresponds to the frequency where the output voltage drops to about 70.7% of its low-frequency value:

$$20\log_{10}\left(\frac{V_{\text{out}}}{V_{\text{in}}}\right) = -3 \text{ dB} \quad \Rightarrow \quad \frac{V_{\text{out}}}{V_{\text{in}}} = 10^{\frac{-3}{20}} \approx 0.707$$

- At 1 kHz, the output waveform amplitude remained almost equal to the input, showing negligible attenuation.
- As the frequency increased, a gradual decrease in amplitude was observed, shown in Fig. 4. The gradual roll-off curve illustrates that higher-frequency components are attenuated, ensuring that only the Doppler frequency remains within the IF bandwidth.
- At 15 kHz, the multimeter measured an attenuation of approximately **–3.18 dB** and the oscilloscope showed a voltage drop from **2.16 V** to **1.6 V** (–2.6 dB), shown in Fig. 5. This confirms a cutoff frequency close to 15 kHz.

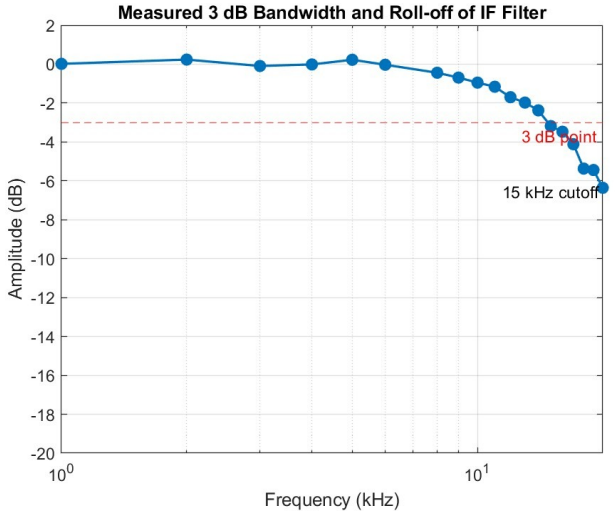
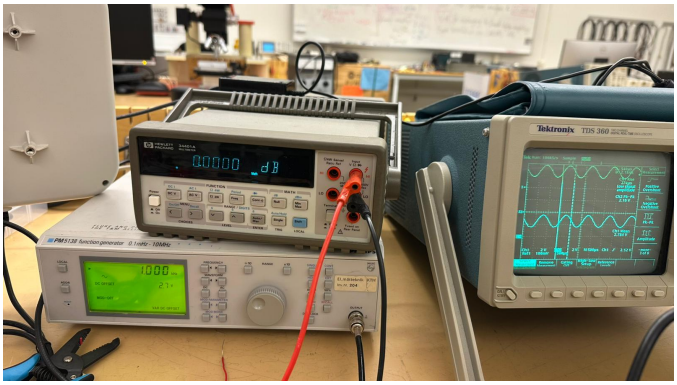
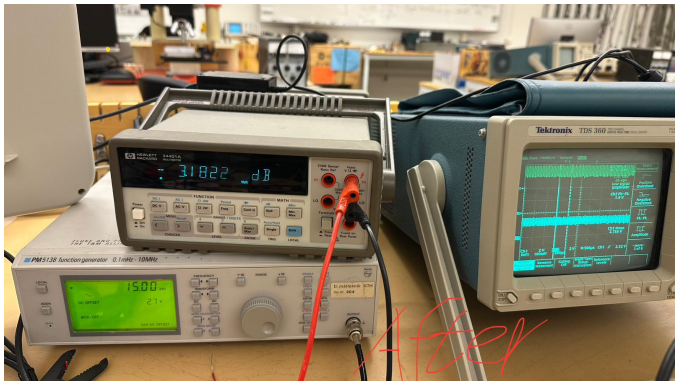


Figure 4: 3dB Roll-off curve of the LPF



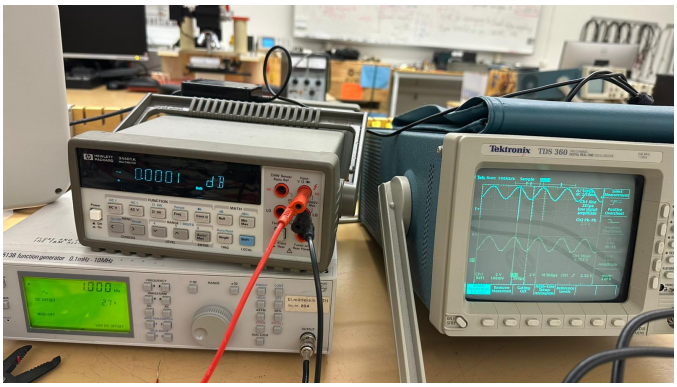
(a) 1 kHz output



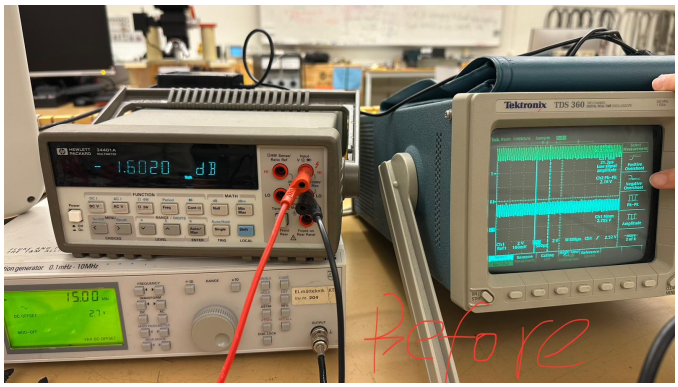
(b) 15 kHz output

Figure 5: 3 dB Roll-off verification (after resistor)

In addition, we repeated the measurement with the probe placed **before the 47 kΩ resistor**, directly at the filter output. In this configuration, the multimeter showed smaller attenuation value (around **–1.6 dB**) and the oscillator showed a voltage drop from **2.4 V** to **2.16 V** (–0.92 dB) at 15 kHz, shown in Fig. 6. Since the voltage divider effect of the probe was eliminated, probing before the resistor risks loading the filter output and disturbing the true signal seen by the sound card input. Therefore, the measurement taken after the resistor is considered more accurate and representative of the system's real performance.



(a) 1 kHz output



(b) 15 kHz output

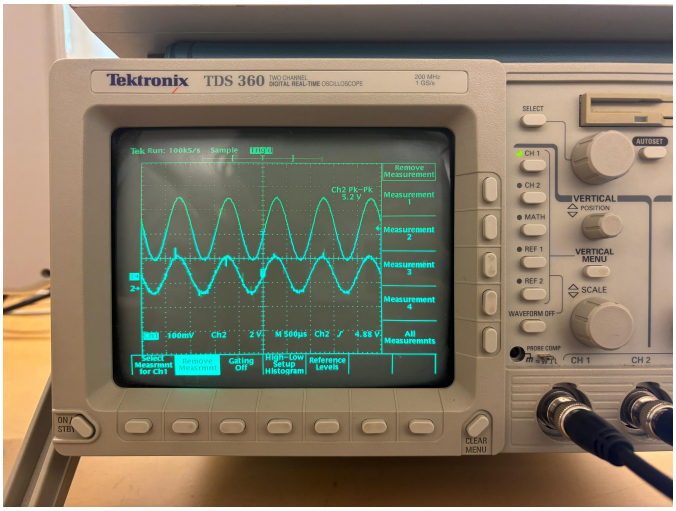
Figure 6: 3 dB Roll-off verification (before resistor)

2.1.4 Gain adjustment

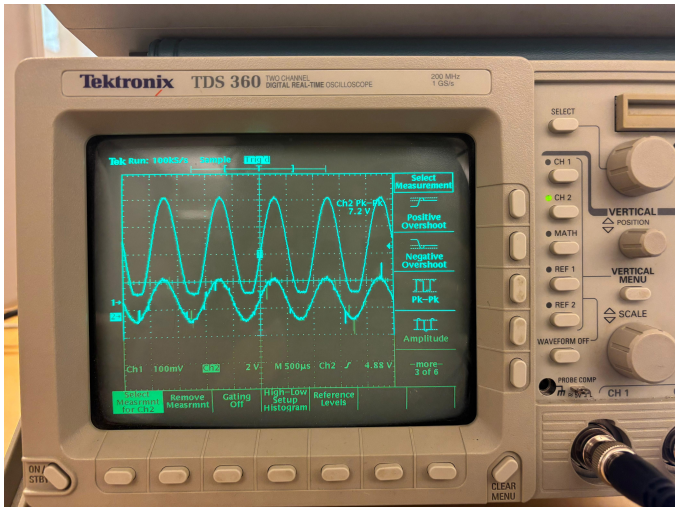
When the gain is set too high, the output signal amplitude exceeds the voltage limits of the video amplifier. On the oscilloscope, this appears as the peaks of the sine wave being "cut off" (flattened) at the top and/or bottom. This distortion is known as clipping, and it must be avoided to preserve signal integrity.

At moderate gain settings, the output waveform remains fully sinusoidal, as shown in Fig. 7(a). However, when the gain is increased beyond the linear range, the peaks of the signal are cut off, as seen in Fig. 7(b). This clipping can corrupt the Doppler information in the radar signal.

For the final configuration, the amplifier trimmer was adjusted until a clean sinusoidal waveform was obtained without any visible clipping.



(a) Without clipping



(b) With clipping

Figure 7: Clipping observation

2.2 Signal Processing Development (MATLAB)

2.2.1 Velocity measurement algorithm (CW mode)

The continuous-wave (CW) velocity measurement algorithm was implemented following the procedure discussed in the course material, and can be found in the appendix in Listing 1. The objective was to estimate the target velocity by analyzing the Doppler frequency shift in the received signal. The recorded audio signal was imported into MATLAB using the `audioread` function, providing the signal data and sampling frequency F_s.

First, the signal was segmented into sweeps of duration T_p, forming a data matrix of dimension $M \times N$, where M is the number of sweeps and $N = T_p F_s$ is the number of samples per sweep. We varied values of T_p between 0.1 s and 1 s to understand its effect on temporal and velocity resolution. Short T_p values provide better temporal resolution, allowing rapid changes in target motion to be tracked, but at the cost of reduced velocity precision, making it harder to distinguish targets with similar velocities. Long T_p values instead improve velocity resolution, giving more precise velocity estimates but reducing temporal resolution: this can make the spectrogram appear pixelated and slow to respond to fast target movements.

We applied Mean Subtraction (MS) clutter rejection on the data, but in our velocity spectrograms no visible difference was observed with or without MS. We assumed it was because the measurement scene contained only weak stationary clutter, resulting in a mean of the entire data matrix close to zero.

A Fast Fourier Transform (FFT) was then computed along each sweep (row), with zero-padding for smoother visualization. Increasing the padding produces smoother plots, making target peaks easier to interpret, while less padding gives coarser results. However, zero-padding does not change the actual resolution, only the visual smoothness of the output. The magnitude of the FFT was then converted to the logarithmic scale using

$$Y_{\text{dB}} = 20 \log_{10}(|Y|).$$

Only the lower half of the FFT output, corresponding to positive Doppler frequencies up to $F_{\max} = F_s/2$, was kept. We then investigated two normalization approaches; in global normalization, the maximum value of the entire matrix is subtracted, so that the strongest point in the spectrogram becomes 0 dB and all other values are relative to it. In row-wise normalization instead, each row corresponding to a time frame is normalized individually by subtracting its row maximum. This sets each row's strongest point to 0 dB, but the relative brightness between frames is no longer meaningful.

In the last step, doppler frequencies f_D were converted to velocities v using

$$v = \frac{\lambda}{2} f_D = \frac{c}{2 f_c} f_D,$$

where c is the speed of light and f_c the radar carrier frequency.
Our results confirmed that larger T_p improved velocity resolution, whereas shorter T_p allowed faster updates. The two normalization methods provided either consistent amplitude reference across time or enhanced contrast between sweeps.

2.2.2 Range measurement algorithm (FMCW mode)

The code for the FMCW range measurement algorithm can be found in Listing 2 in the appendix. The recorded audio signal was first separated into sync and backscatter channels. The backscatter data corresponding to up-chirps was segmented into a matrix for processing, with each row representing a sweep. To suppress stationary clutter, MS clutter rejection was applied across each column of the matrix. Without MS, strong stationary clutter close to the radar (e.g., 0–20 m) dominates the range spectrum, and its sidelobes mask weaker moving targets. With the application of MS the mean static ridge is removed, lowering the background floor and improving visibility of weaker targets.
The range resolution is determined by the sweep bandwidth Δf, and the range of each FFT bin is given by

$$\Delta R = \frac{c}{2\Delta f}, \quad R_{\max} = \frac{N\Delta R}{2},$$

where c is the speed of light and N is the number of samples per sweep. FFT zero-padding was applied to interpolate the frequency grid, because as previously mentioned it provides smoother plots for visual interpretation.
To enhance detection of moving targets, 2-pulse and 3-pulse MTI clutter rejection was applied. In 2-pulse MTI, consecutive sweeps are subtracted:

$$\mathrm{MTI}_2[n] = x[n+1] - x[n],$$

while 3-pulse MTI subtracts a combination of three sweeps:

$$\mathrm{MTI}_3[n] = x[n+2] - 2x[n+1] + x[n].$$

Without MTI, stationary clutter dominates the output, making moving targets difficult to detect. 2-pulse MTI reduces this clutter but leaves some low-Doppler residuals. 3-pulse MTI further suppresses slow-moving clutter, improving moving target visibility, though very slow targets may be slightly attenuated.

2.2.3 Range–Doppler measurement algorithm

The range-Doppler algorithm, found in Listing 5 was implemented to jointly estimate target range and radial velocity from the FMCW data. Each sweep was first processed in range using FFT across samples, and then Doppler analysis was performed by applying a second FFT across multiple sweeps. The resulting matrix represents power as a function of both range and Doppler frequency, providing a two-dimensional map of target motion. The range-Doppler map was computed as

$$Y_{\mathrm{RD}}(r, f_D) = \mathrm{FFT}_{\mathrm{Doppler}}\{\mathrm{FFT}_{\mathrm{Range}}\{x(t)\}\}.$$

The magnitude of Y_{RD} was converted to decibel scale as $Y_{\mathrm{dB}} = 20\log_{10}(|Y_{\mathrm{RD}}|)$ for visualization.
Without MS clutter rejection and 2-pulse MTI, stationary clutter dominates the range-Doppler map. When MS and 2-pulse MTI are applied, the static and low-Doppler clutter components are largely removed, revealing the moving targets clearly and enhancing detection clarity. With MS only, the clutter level decreases but low-Doppler pixelation remains visible, reducing target contrast. The Doppler resolution depends on the time duration T_{chunk} of the processed data segment, given approximately by

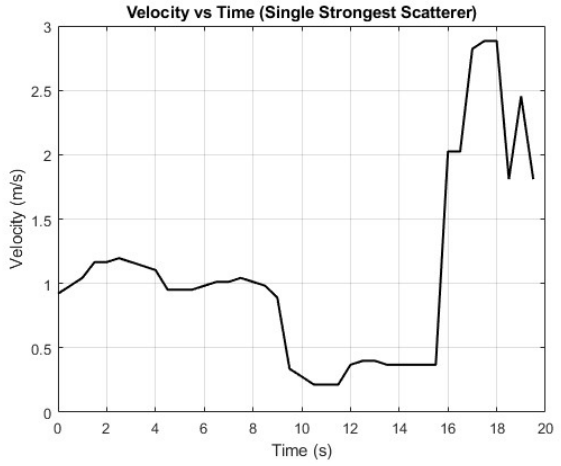
$$\Delta f_D = \frac{1}{T_{\mathrm{chunk}}}.$$

Shorter chunks (less than 1 s) provide faster updates but coarser Doppler resolution, making targets appear broader and less distinct. Longer chunks (greater than 1 s) improve Doppler precision, leading to sharper peaks but at the cost of temporal responsiveness.

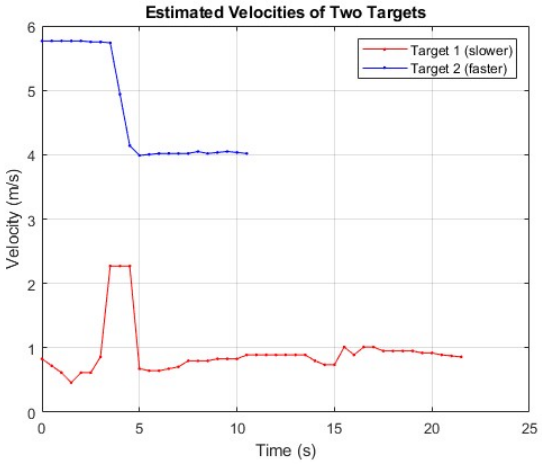
2.3 Experimental Measurements

2.3.1 Velocity measurement(CW mode)

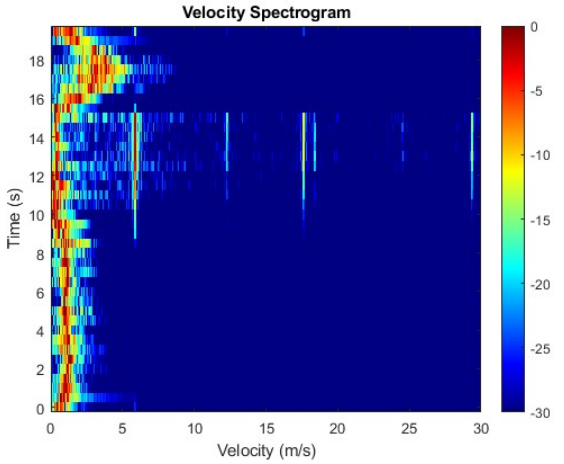
Next we conducted measurements using the 2.4 GHz COTS radar setup. We performed velocity-over-time measurements for both single and multiple targets, and we used the previously developed signal processing algorithms for CW mode. For single-target experiments, the algorithm was applied as in earlier simulations, with parameter adjustments accounting for the measured sampling rate and sweep period. For multiple-target experiments, the signal processing algorithm was modified to identify and track the two strongest scatterers in each frame. This approach enables effective separation of dominant reflectors corresponding to different moving targets while maintaining temporal coherence across sweeps.
In the single-target experiment, the motion pattern consisted of three phases. The target first moved away from the radar at a walking speed, then paused briefly, and finally returned towards the radar at a running speed. This motion pattern can be seen in Fig. 8a and is reflected in the corresponding spectrogram in Fig. 8c. The plot captures only positive velocity magnitudes, regardless of whether the target moves toward or away from the radar.
In the multiple targets experiment, both targets started moving simultaneously. One target moved away from the radar at a running speed and stopped after reaching a certain distance, while the other one moved towards the radar at a walking speed, stopping only upon reaching the setup, walking for a longer time period: this resulted in two distinct Doppler components corresponding to different target velocities. From Fig. 8b and Fig. 8d, it can be observed that the first target (the runner) moves at a higher velocity and stops around second 10, while the walking target continues moving at a lower speed until approximately second 22.



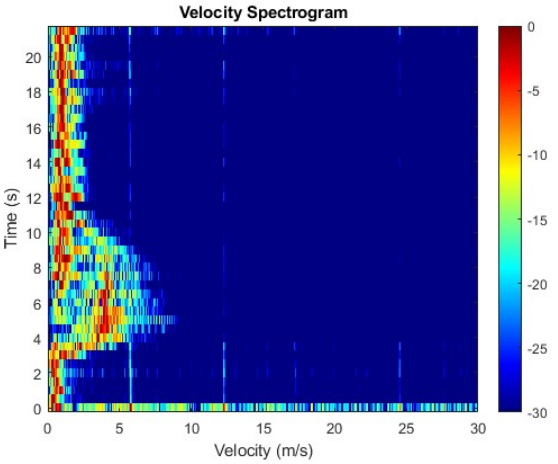
(a) Velocity vs Time plot for single target



(b) Velocity vs Time plot for multiple targets



(c) Velocity vs Time spectrogram for single target



(d) Velocity vs Time spectrogram for two targets

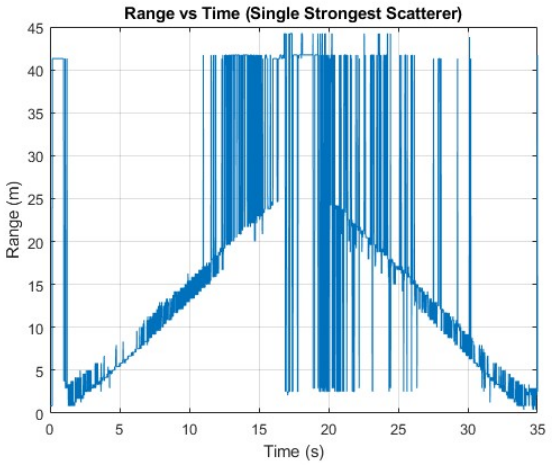
Figure 8: Plots for CW measurements with COTS radar

2.3.2 Range measurement(FMCW mode)

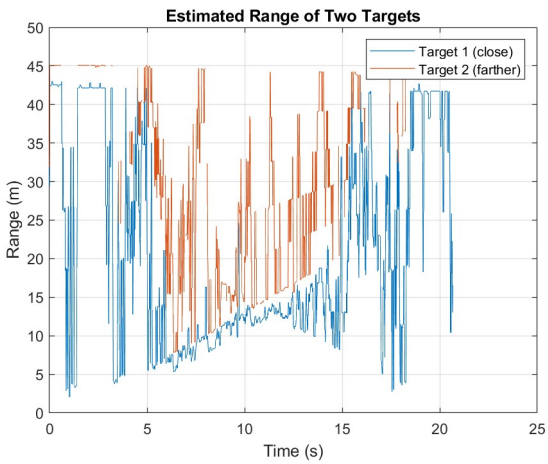
We then switched to FMCW mode to conduct range-over-time measurements. The previously developed FMCW signal processing algorithm was applied, with modifications to account for the measured sampling rate, sweep duration, and the need to handle multiple targets. Mean Subtraction (MS) clutter rejection was applied to suppress static clutter and we tried using both 2-pulse and 3-pulse MTI to enhance moving target detection. For single-target experiments, the algorithm followed the standard procedure, estimating the range of the strongest scatterer in each sweep. For multiple-target experiments, the two strongest peaks in each sweep were identified and tracked.

In the single-target experiment, the target moved away from the radar along a straight line at a walking speed, paused for a few seconds, and then returned along the same path at a walking speed. This motion pattern is reflected in the range-over-time plot Fig. 9a and spectrogram Fig. 9c. The target appears to be moving and increasing range during the outbound motion, "disappears" during the pause, and then decreases in range during the return. Plotting local maxima revealed that increasing the detection threshold reduces noise in the range plot but risks missing the target at longer distances, while lower thresholds allow detection throughout the range but result in noisier plots. The range estimates were then smoothed using a moving average to improve readability and to compensate for small fluctuations in the detected peaks.

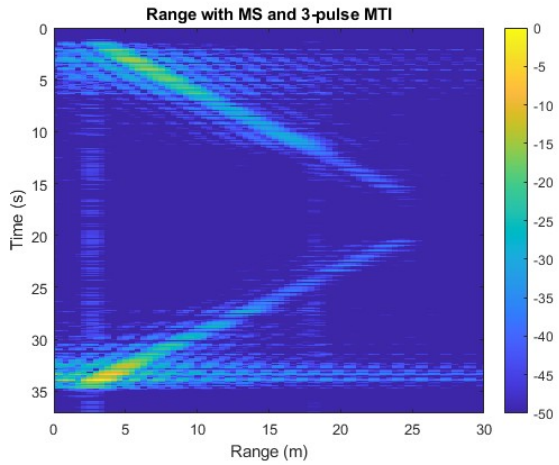
In the two-target experiment, both targets started moving simultaneously at walking speed. One target moved away from the radar while the other moved towards the radar, both moving along a straight path. This generated two distinct range components corresponding to the two targets: to track both of them, the two largest local maxima were plotted for each sweep. As in the single-target case, increasing the detection threshold reduces peaks but can cause the second target to be missed, whereas a lower threshold ensures detection at the cost of noisier and less stable tracking, with occasional jumps between targets. This can be observed in Fig. 9b and Fig. 9d below.



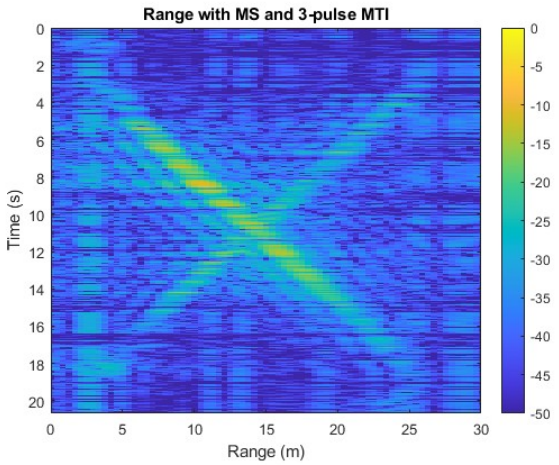
(a) Range vs Time for a single target



(b) Range vs Time for multiple targets



(c) Range vs Time spectrogram for a single target

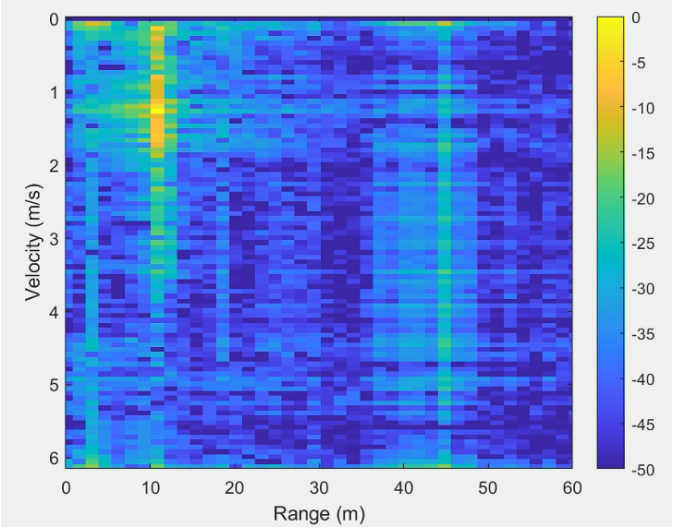


(d) Range vs Time spectrogram for two targets

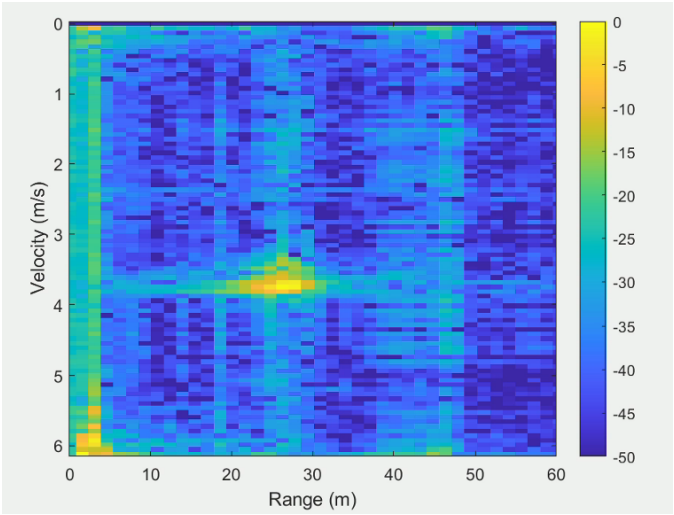
Figure 9: FMCW measurement results with COTS radar.

2.3.3 Range-Doppler measurement

The Range-Doppler maps were obtained by applying FFT processing in both fast-time (range) and slow-time (Doppler) domains. During the measurement, the target moved away from the radar along a straight line at a walking speed until reaching a certain distance. It then remained stationary for a few seconds before returning along the same path at a running speed. This motion pattern allowed both positive and negative Doppler shifts to be observed, corresponding to the target moving away from and toward the radar, respectively. Two spectrograms at different target positions reflected clear changes in both range and velocity, shown in Fig. 10 below.



(a) Range–Doppler map at position 1



(b) Range–Doppler map at position 2

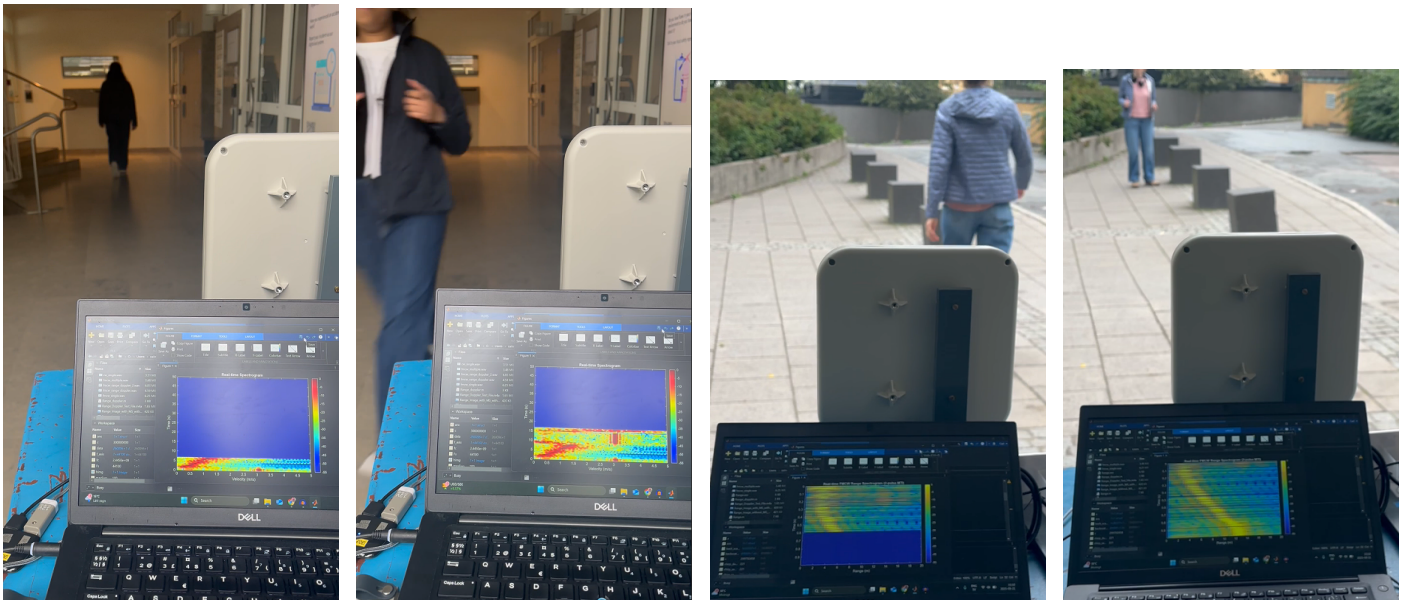
Figure 10: Range–Doppler spectrograms at two different target positions

2.3.4 Real-time measurements for velocity in CW mode

In CW mode, real-time Doppler processing was implemented using short-time FFT to display the velocity variation continuously. The audio was acquired and processed in real time directly in MATLAB, rather than being recorded with Audacity and processed afterward. In the measurement, the target first moved away from the radar at a walking speed. It then paused for a few seconds before returning toward the radar at a running speed. This motion produced both positive and negative Doppler frequency shifts, clearly observable in the real-time velocity spectrogram. Two spectrograms at two different positions reflected changes in target speed, shown in Fig. 11 (a)(b).

2.3.5 Real-time measurements for range in FMCW mode

In FMCW mode, the beat frequency was processed in real time to update the range profile continuously. The audio was acquired and processed in real time directly in MATLAB, similar to real-time velocity measurements. In the measurement, the target moved away from the radar along a straight line at a walking speed until reaching a certain distance. It then returned along the same path at the same walking speed. This motion caused the detected range peak to first increase and then decrease over time, demonstrating the radar's ability to track target distance dynamically. Two spectrograms at two different positions reflected changes in target range, shown in Fig. 11 (c)(d).



(a) CW at Position 1 (b) CW at Position 2 (c) FMCW at Position 1 (d) FMCW at Position 2

Figure 11: Real-time radar measurements at two target positions

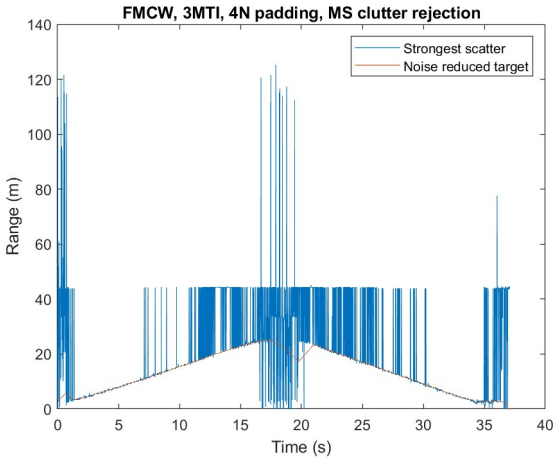
2.3.6 Extract velocity from range using two-point finite difference method for a single target

Since we have several range measurements of a target over a time period we can differentiate that data and thereby get velocity measurementsref. We differentiate it using the two-point finite difference method;

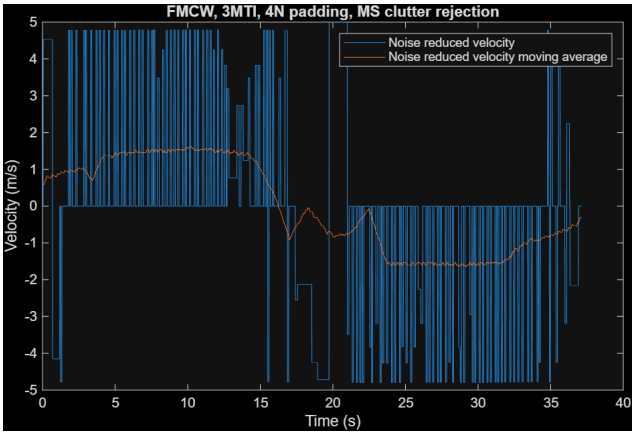
$$v = \frac{x(n+1) - x(n)}{t(n+1) - t(n)}$$

where v is the velocity, x is the position of the target, and t is the time.

We apply this formula to the data from Fig. 12a. Because of our limited range resolution the resulting velocity oscillates very fast. To get the final velocity estimate we apply a moving average to this data. The results are plotted in Fig. 12b.



(a) Target range



(b) Extracted velocity

Figure 12: Two-point finite difference method

2.4 SAR Imaging

2.4.1 SAR Time Domain Correlation for simulated targets

The Time-Domain Correlation (TDC) algorithm is a fundamental Synthetic Aperture Radar (SAR) imaging approach that reconstructs the scene reflectivity by coherently summing the backscattered echoes over both radar positions and fast-time samples. TDC operates directly in the time domain, making it easy to implement but computationally more demanding for SAR image focusing. The reflectivity function $f(x, z)$ is computed as

$$f(x,z) = \sum_{x'} \sum_{t} S_b(x',t)\, e^{j2\pi\left(f_c\,\tau(x',x,z)+\frac{B}{T_p}\,\tau(x',x,z)\,t\right)},$$

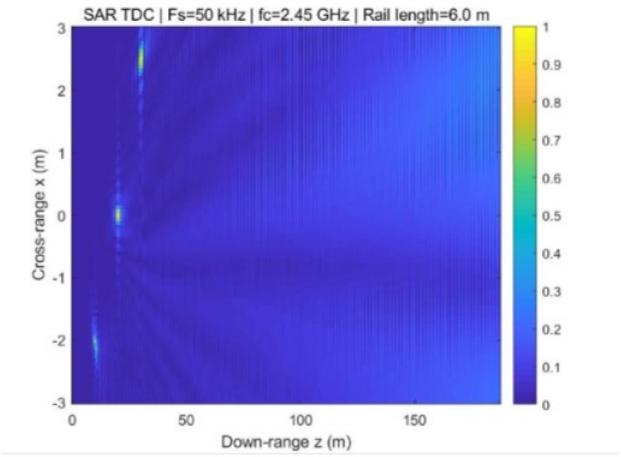
where $S_b(x', t)$ is the baseband signal, f_c the carrier frequency, B the bandwidth, T_p the chirp duration, and $\tau(x', x, z) = \frac{2}{c}\sqrt{(x - x')^2 + z^2}$ the round-trip delay from radar position x' to pixel (x, z). After correlation, a range-squared (R^2) compensation is applied to account for spherical spreading loss.
To analyze system performance, we generated simulated data and we studied the effects of key parameters on image quality and computational performance as follows:

1. Sampling frequency (F_s): Increasing F_s from 50 kHz to 250 kHz increases the number of fast-time samples ($N = F_s T_p$), leading to longer computation times since every sample is processed at each pixel. However, range resolution and image sharpness remain unchanged, while the maximum observable range increases proportionally to N.

2. Rail length (L_{aperture}): Doubling the aperture length improves cross-range resolution, as it is inversely proportional to L, i.e., $\rho_x \approx \frac{\lambda R}{2L}$. Targets appear narrower and better resolved in cross-range, while computation time increases moderately due to more aperture positions.

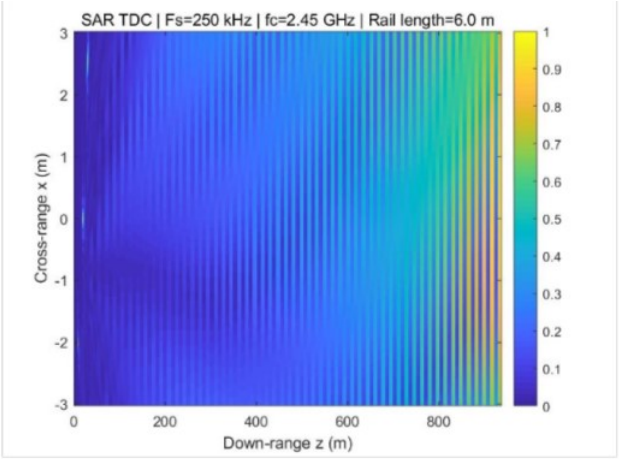
3. Target range position (R): The targets located farther from the radar show worse cross-range resolution, consistent with $\rho_x \propto R$. At longer ranges, the target responses become broader and less sharp.

4. Center frequency (f_c): Increasing the center frequency (e.g., from 2.45 GHz to 5.45 GHz) improves the cross-range resolution by roughly a factor of two, since λ is halved and $\rho_x \approx \lambda R/(2L)$. As a result, the targets appear much sharper in cross-range. The computation time increases slightly because the spatial sampling spacing becomes smaller, leading to more radar positions along the rail. The maximum measurable range remains essentially unchanged.



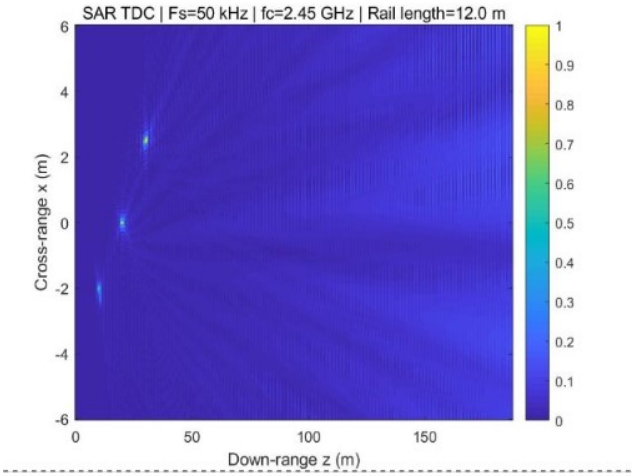
Processing time: 3.898 s
Range resolution ΔR = 1.50 m, Max range Rmax = 187.5 m (N=250)
Cross Range Resolution = 0.20 m

(a) Baseline case



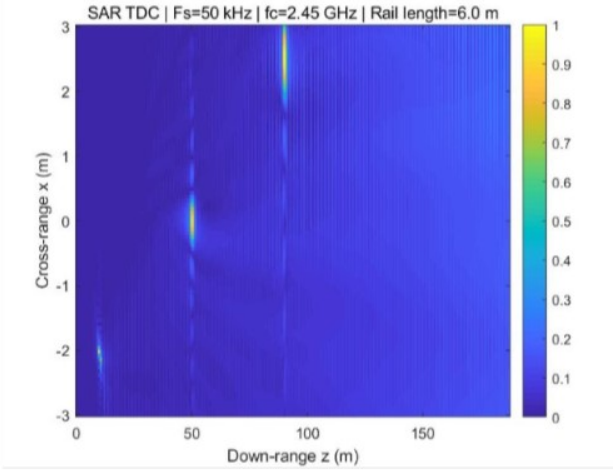
Processing time: 270.946 s
Range resolution ΔR = 1.50 m, Max range Rmax = 937.5 m (N=1250)
Cross Range Resolution = 0.20 m

(b) Case 1: Fs = 250 kHz



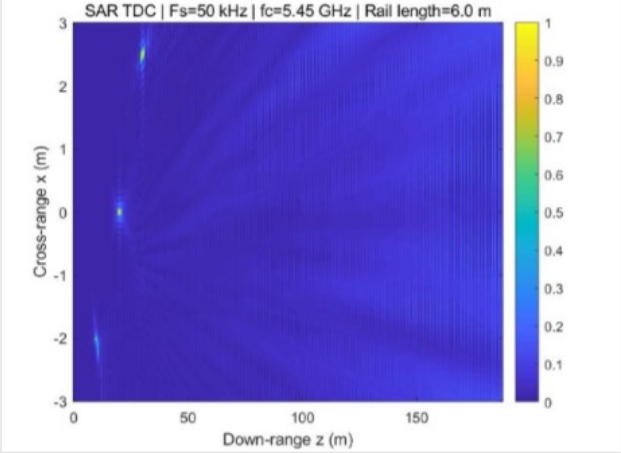
Processing time: 13.481 s
Range resolution ΔR = 1.50 m, Max range Rmax = 187.5 m (N=250)
Cross Range Resolution = 0.10 m

(c) Case 2: Rail length = 12 m



Processing time: 4.042 s
Range resolution ΔR = 1.50 m, Max range Rmax = 187.5 m (N=250)
Cross Range Resolution = 0.51 m

(d) Case 3: Targets at [50, 50, 90] m



Processing time: 17.090 s
Range resolution ΔR = 1.50 m, Max range Rmax = 187.5 m (N=250)
Cross Range Resolution = 0.09 m

(e) Case 4: Higher frequency 5.4–5.5 GHz

Figure 13: SAR images generated using Time-Domain Correlation (TDC) for simulated targets

In the plots in Fig.13 above, Fig. 13a represents the baseline, with $F_s = 50$ kHz, rail length 6 m, target positions [10, 20, 30] m and $f = 2.4$–2.5 GHz. Figure 13b uses a higher sampling frequency $F_s = 250$ kHz, while figure 13c corresponds to a longer rail length of 12 m. Figure 13d shows the effect of placing targets farther away at [50, 50, 90] m and lastly, figure 13e shows the case with increased radar frequency $f = 5.4$–5.5 GHz.

2.4.2 Range Migration Algorithm for simulated targets

The Range Migration Algorithm (RMA) is a frequency-domain Synthetic Aperture Radar (SAR) processing technique used to focus radar images of targets while compensating for range migration due to the radar's motion. RMA transforms raw SAR data into a focused image using the following key steps:

1. Conversion to frequency–wavenumber domain via 2D FFT.
2. Stolt interpolation to linearize range-phase relationship and correct migration.
3. 3D inverse FFT to obtain the reflectivity function $c(x, y, z)$.

The equations used in obtaining the parameters and implementing the steps are listed as follows, The dispersion relation connects temporal and spatial frequencies:

$$k_z = \sqrt{4k_t^2 - k_x^2 - k_y^2},$$

where

$$k_t = \frac{2\pi f_c}{c} + \frac{2\pi B}{T_p} t,$$

and k_x, k_y are spatial wavenumbers. Once we have the spatial wave-numbers the SAR baseband data $S_b(x', y', k_t)$ is transformed as

$$\tilde{S}_B(k_x, k_y, k_t) = \mathcal{F}_{2D}\{\tilde{S}_b(x', y', k_t)\}.$$

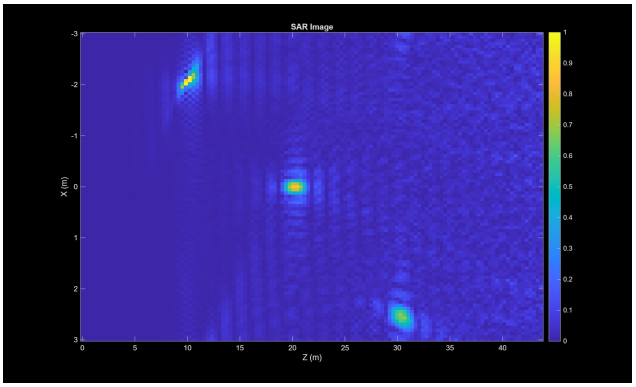
Once we have the FFT data, we apply Stolt mapping to correct for non-linear migration as,

$$\tilde{S}_B(k_x, k_y, k_t) \rightarrow \tilde{S}_B(k_x, k_y, k_z).$$

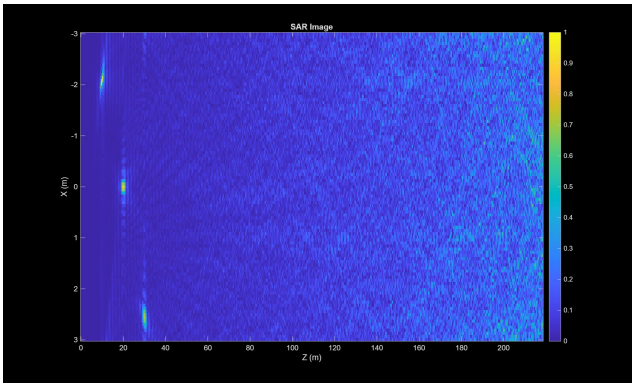
After the interpolation, the final focused image is obtained using a 3D inverse FFT as

$$c(x, y, z) = \mathcal{F}_{3D}^{-1}\{\tilde{S}_B(k_x, k_y, k_z)\}.$$

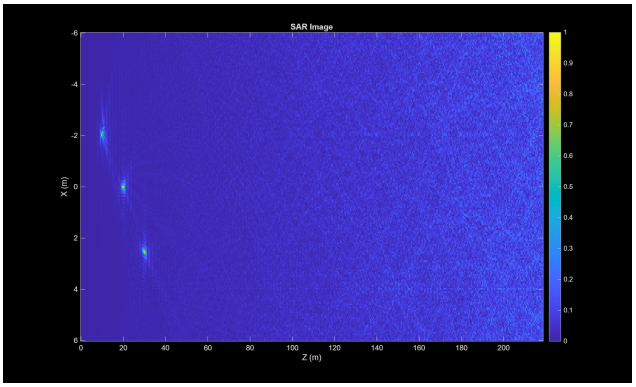
The performance of RMA-based SAR radar is critically influenced by key system parameters, where balancing resolution and computational efficiency is essential. Here, we tested different parameters on simulated targets to see the effects. Firstly, increasing the sampling frequency (F_s) from 50 kHz to 250 kHz results in only marginally increased measured total runtime, i.e., from 0.564 s to 0.584 s. This indicates that for this specific problem size, the computational load is dominated by other processing steps rather than the data volume. Regarding spatial resolution, the synthetic aperture length (L_{aperture}) plays an important role. Extending the rail from $x_p = -3{,}000$ mm to 3,000 mm to a longer range of $x_p = -6{,}000$ mm to 6,000 mm increased the total generation time only slightly, from 0.584 s to 0.597 s, indicating that the added data volume has minimal impact on overall processing time. However, the longer rail noticeably improves the cross-range resolution, as seen in Figure 14c, where the targets appear less stretched. On the other hand, the target range position (R) negatively impacts resolution; targets situated further away exhibit a worse cross-range resolution due to the increased slant-range curvature, which can be observed from figure14d. Finally, increasing the radar center frequency further enhances image sharpness but introduces a clear trade-off. Using a higher frequency takes 0.609 s to generate the image and results in improved cross-range resolution; however, it simultaneously reduces the maximum unambiguous range.



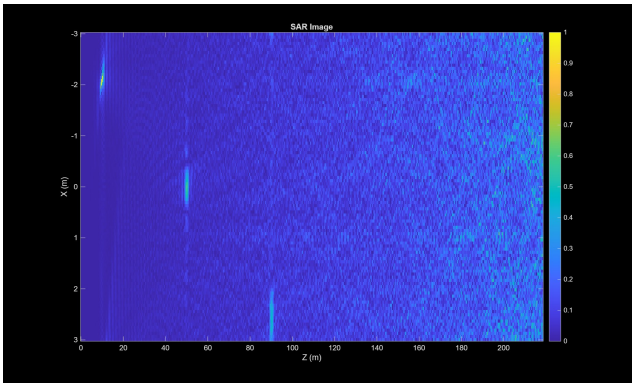
(a) Results with Fs = 50KHZ



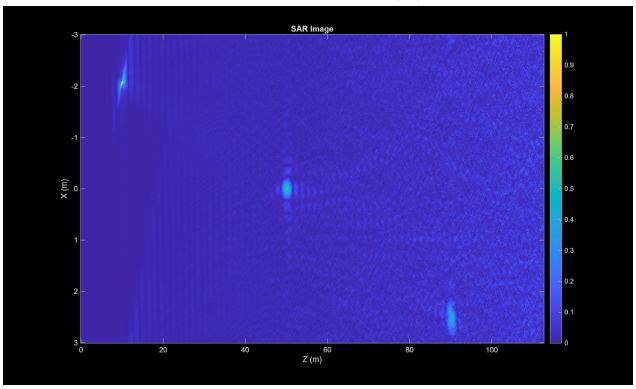
(b) Results with Fs = 250KHz



(c) Results of using a longer rail length



(d) Results when targets are placed further away



(e) Results of using a higher radar frequency (5.4-5.5GHz)

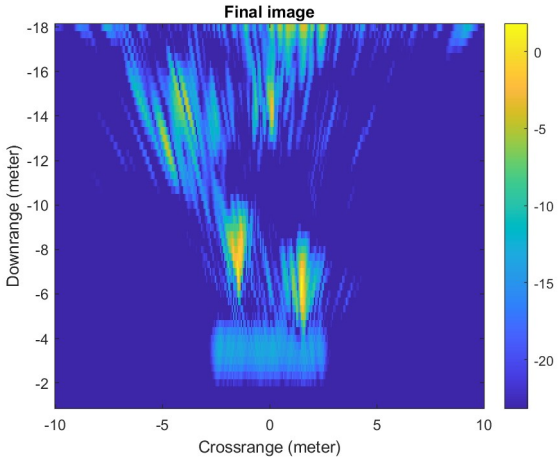
Figure 14: Effects of system parameters on SAR image processing using RMA on simulated targets

2.4.3 Range Migration Algorithm for real targets

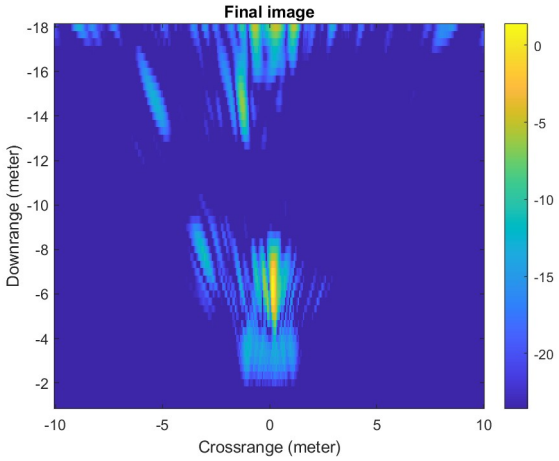
Processing real radar data using the RMA requires three distinct steps that are not typically needed for simulated targets. First, the initial raw data is divided based on the number of positions we have taken measurements from. Then we extract the chirps from each position, integrate and average the corresponding consecutive chirps (divided by the number of chirps per position) to manage data size and reduce noise. This process creates the final sparse data set, which is then used to create the physical downrange and cross-range vectors, based on measured radar parameters, to correctly scale the final image in meters. Secondly, a 2D windowing function (such as Hann or Hamming) must be applied to the raw data before the 2D FFT to suppress the high sidelobes that cause blurring. Finally, the compensated data is scaled to the logarithmic decibel (dB) scale for visualization, which effectively displays the wide dynamic range of real-world target reflectivity. Similar to the processing in simulated targets, a critical radiometric correction is performed by multiplying the processed data by the square of the downrange vector (R^2 compensation), which counters the $1/R^2$ spherical spreading loss.

2.4.4 Practical SAR Measurement

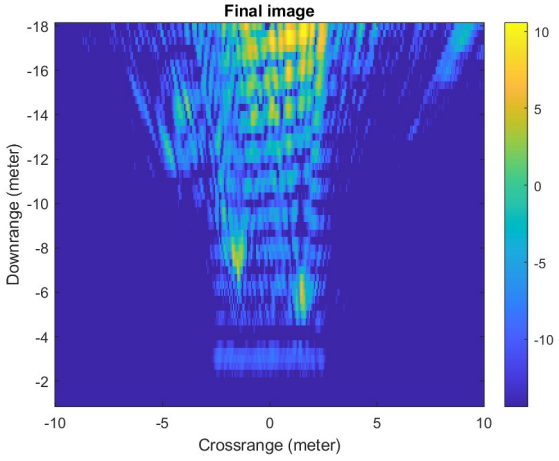
The SAR measurements were performed in FMCW mode using a rail-based setup. A single table segment of approximately 60cm was used for the short rail configuration, while three connected tables (about 1.8m) formed the long rail configuration. Three trihedral corner reflectors were placed within 20 m of the radar at different positions, which served as stationary targets. The radar was moved in 6cm steps, corresponding to approximately $\lambda/2$ at 2.5GHz along the rail, and at each position, 5–10 seconds of data were recorded. The sync switch was toggled on and off between movements to ensure proper synchronization of measurements across positions. The measured data was then processed with the RMA algorithm to get results.



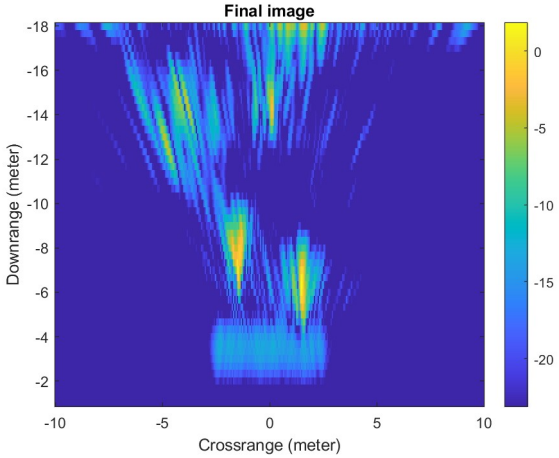
(a) SAR image generated with three table lengths.



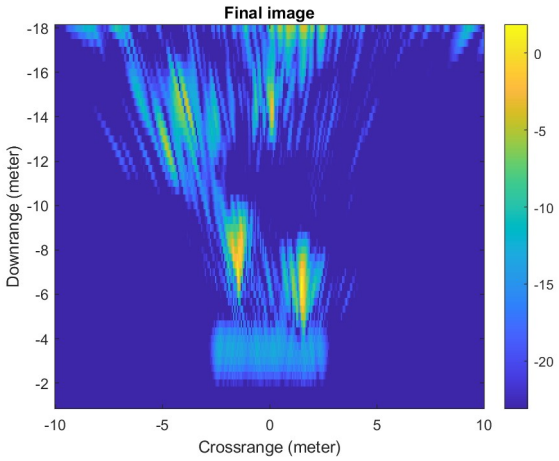
(b) SAR image generated with one table length.



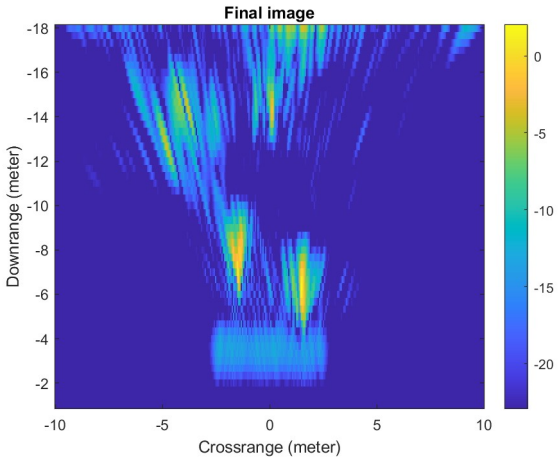
(a) SAR image generated without applying Hann window.



(b) SAR image with $\tau_{rp} = 0.25$.



(a) SAR image with $\tau_{rp} = 0.5$.



(b) SAR image with $\tau_{rp} = 1.0$.

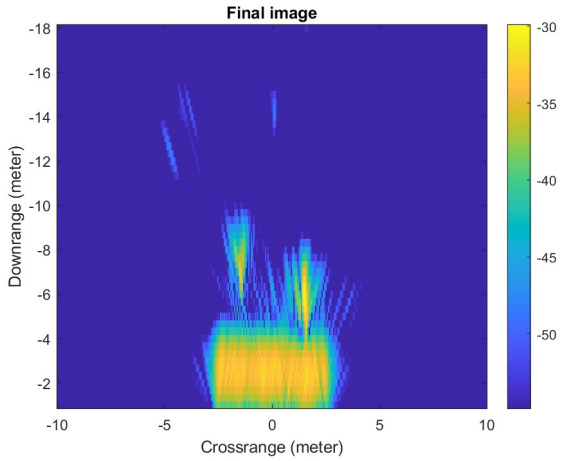


Figure 18: SAR image generated without downrange multiplication.

The final SAR image (Figure 15a), generated using the three table lengths, successfully resolved three distinct targets. The image clearly shows that each target is accurately positioned, confirming the positional accuracy of the reconstruction. As shown in the figure, the bright spot corresponding to the trihedral reflector is well-localized and almost as wide as the physical reflector itself. While strong reflections from a nearby building are visible, the targets are separated. The importance of aperture length is demonstrated by the short-rail processing (Figure 15b), which yielded poorer cross-range resolution, often blurring the three targets into one or two due to insufficient synthetic aperture. Furthermore, we checked for the effects of applying a Hann window. We can observe from Figure 16a that with no windowing, the image results in high-energy sidelobes, indicating spectral leakage, highlighting the necessity of windowing for image quality. The impact of the integration time per position, τ_{rp}, on the signal-to-noise ratio and image quality is noted in figures 16b, 17a & 17b. We can see that with higher τ_{rp}, we have better suppression of background noise (better SNR). Finally, the importance of radiometric correction is shown, as the image without R^2 compensation (Figure 18) displayed a rapid drop-off in target intensity with increasing range, making distant targets appear significantly less bright (weaker) than they are in reality.

3 Software Defined Radar at 5.8 GHz

The 5.8 GHz software-defined radar (SDR) implements the entire transmit/receive chain and signal processing in software, using a wideband RF front end and GNU Radio for waveform generation, data acquisition, and real-time/offline processing. Unlike the earlier 2.4 GHz COTS radar built from discrete analog blocks the SDR platform is reconfigurable by design: carrier, bandwidth, sampling, and

algorithms can be changed in software without rewiring the hardware. This flexibility lets us prototype CW, Low-IF CW, and advanced processing (e.g., IQ demodulation and direction-aware velocity) on the same hardware.

The SDR radar front end uses the Mini-Circuits ZX60-83LN+ low-noise amplifier operating from 0.5 to 8 GHz. To verify its RF performance, the S-parameters (S11, S21, and S12) were measured. The measured curves are shown in Fig. 19. These results demonstrate that the amplifier provides high gain **(20.6 dB)** and stable operation with proper input/output matching at the target frequency, which is critical for accurate signal amplification in the SDR radar chain.

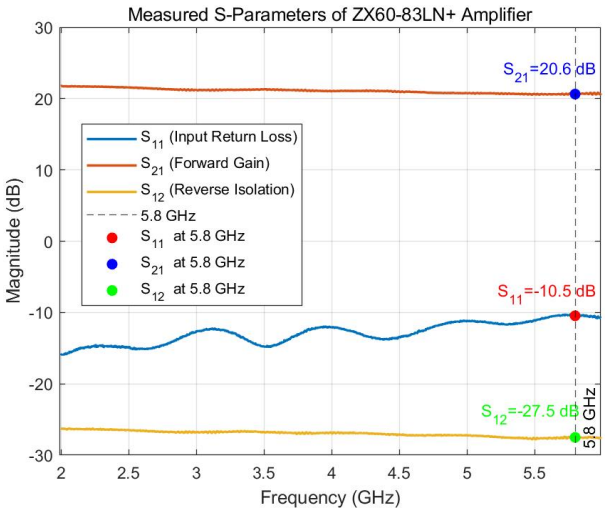


Figure 19: S parameters plot of the amplifier

3.1 Velocity Measurements (without PA)

We measured target velocity at 5.8 GHz with the SDR in two configurations, namely CW and Low IF CW, both operated without a power amplifier. In CW mode, the source sits at 0 Hz (baseband). In Low-IF CW, we offset it to 4 kHz, shifting the receiver's DC noise to ∼4 kHz, i.e. Dopplers for unrealistically high velocities (outside the region we care to measure). We captured the time domain returns, estimated velocity as a function of time from the Doppler frequency, and generated spectrograms using global maximum normalization.

We use the same GNU Radio flowgraph for CW and Low IF CW mode, only changing the signal source frequency. When using IQ demodulation we use both the real and the imaginary output, and when we don't use IQ demodulation we only use the real output.

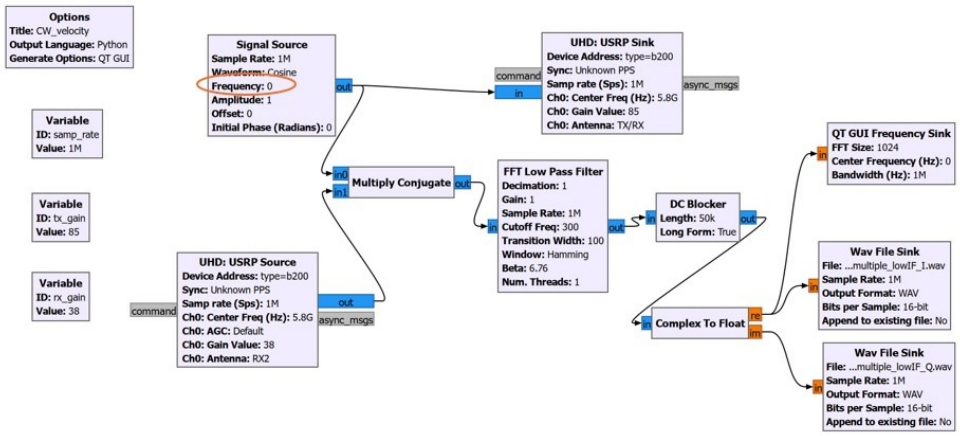
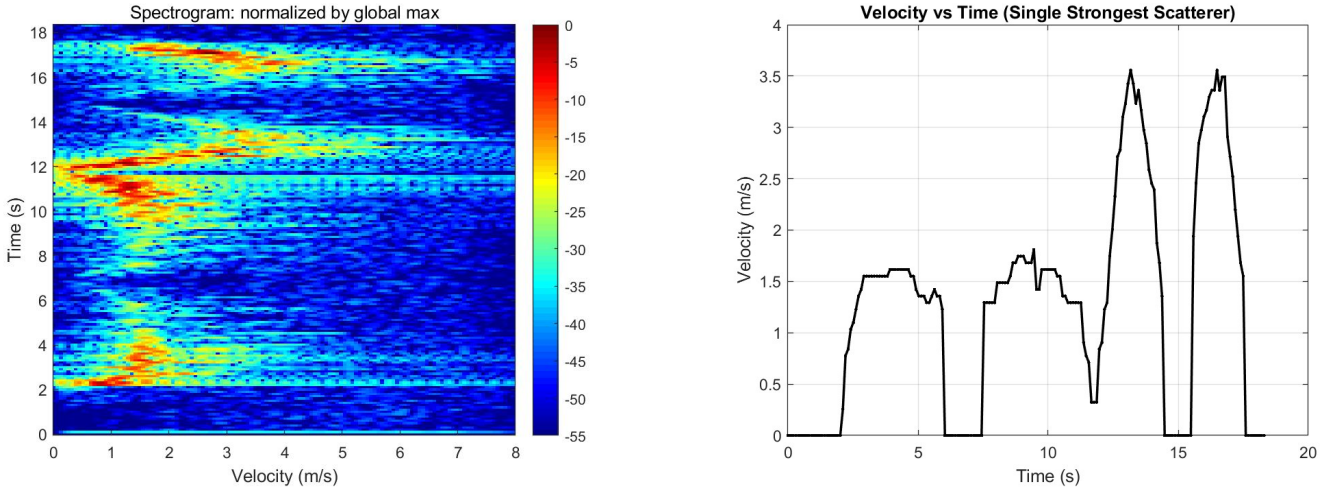


Figure 20: GNU Radio flowgraph

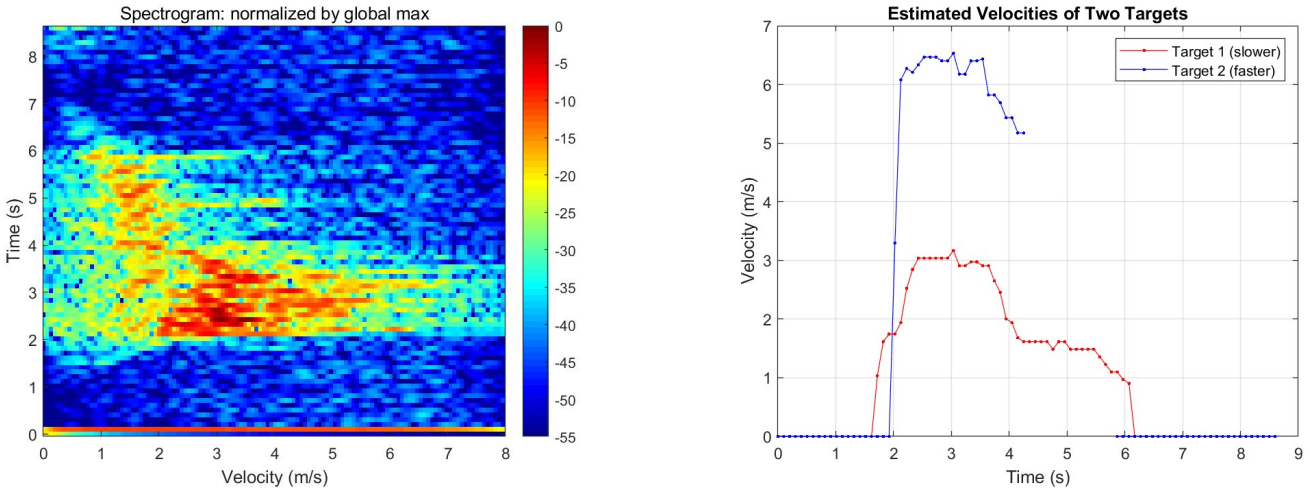
For single-target measurements, one subject walked away slowly from the radar and returned, then ran away fast and returned. For multi-target measurements, one subject ran away fast from the radar while another walked slowly toward it simultaneously. From the plots, we see the radar correctly captures their velocity magnitudes, and that using low IF reduces the noise (especially noticeable when comparing 21(a) and 23(a)). This is because in Low-IF CW mode we operate at 4kHZ, thereby eliminating the Local Oscillator (LO) leakage noise and the associated DC offset spike in the spectrogram.



(a) Velocity vs time spectrogram

(b) Velocity vs time plot

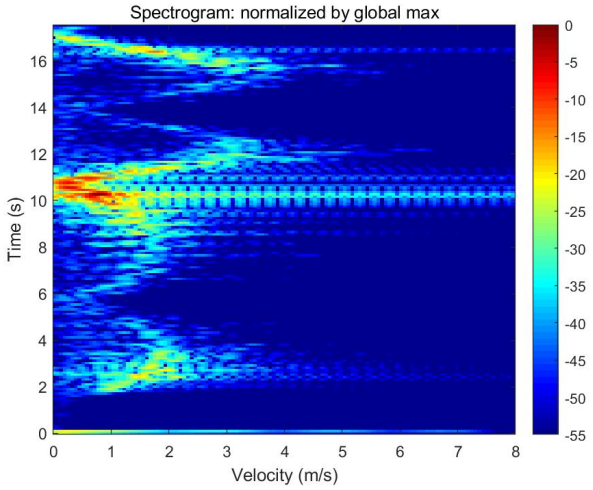
Figure 21: CW mode - Single target



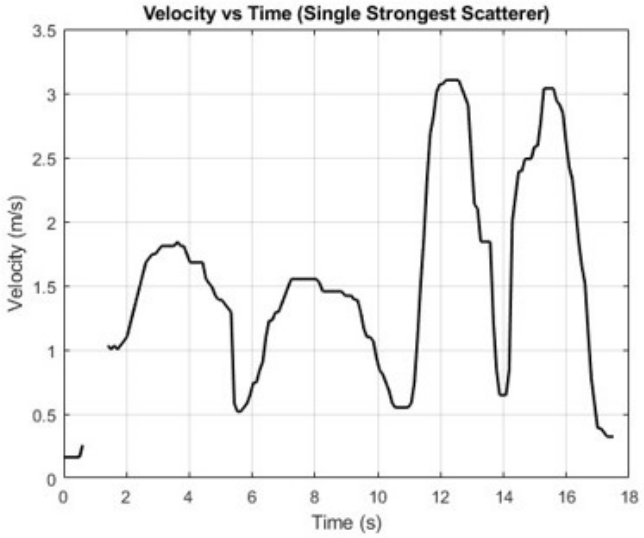
(a) Velocity vs time spectrogram

(b) Velocity vs time plot

Figure 22: CW mode - Multiple targets

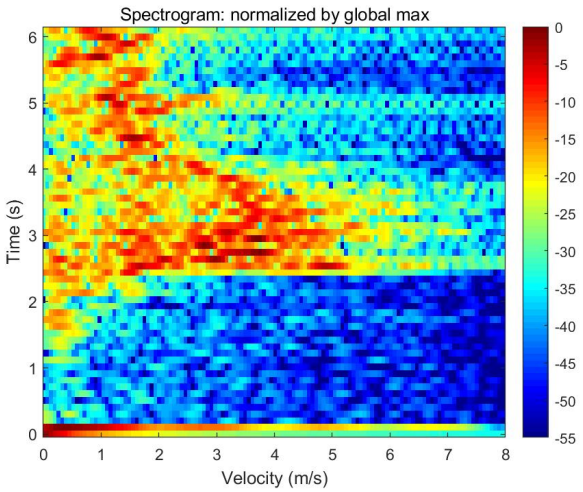


(a) Velocity vs time spectrogram

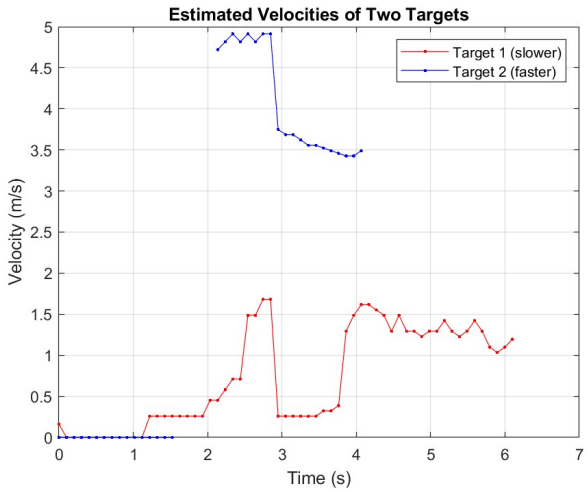


(b) Velocity vs time plot

Figure 23: Low IF CW mode - Single target



(a) Velocity vs time spectrogram



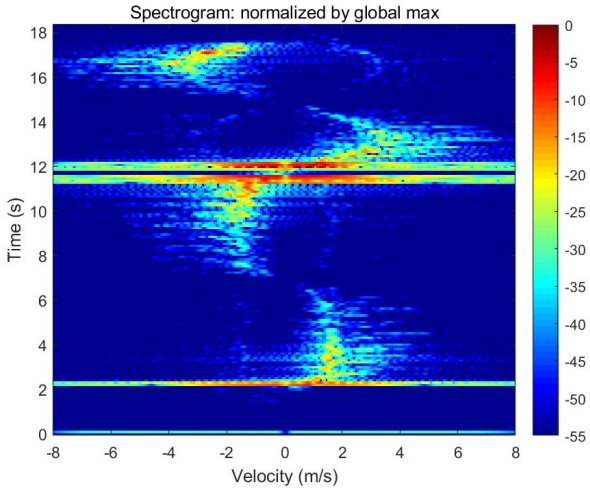
(b) Velocity vs time plot

Figure 24: Low IF CW mode - Multiple targets

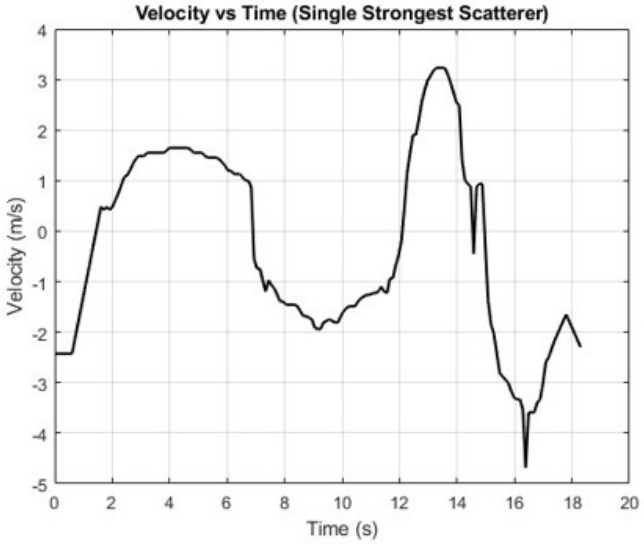
3.2 IQ Demodulation for Direction Detection

To resolve the toward/away ambiguity inherent in real-only CW processing, we enable IQ demodulation so the complex baseband preserves phase and thus the sign of the Doppler shift. We evaluate the approach in both CW and Low-IF CW modes for single and multiple targets using the same GNU Radio flowgraph with complex branches enabled, with results summarized in Figs. 23–26.

In IQ demodulation, the received RF is mixed with two local-oscillator signals 90° apart (cosine and sine), low-pass filtered to produce I and Q, and combined as a complex signal $s(t) = I + jQ$. This analytic signal retains phase rotation direction, so positive/negative Doppler shifts map to forward/backward complex rotation, directly indicating motion toward or away.

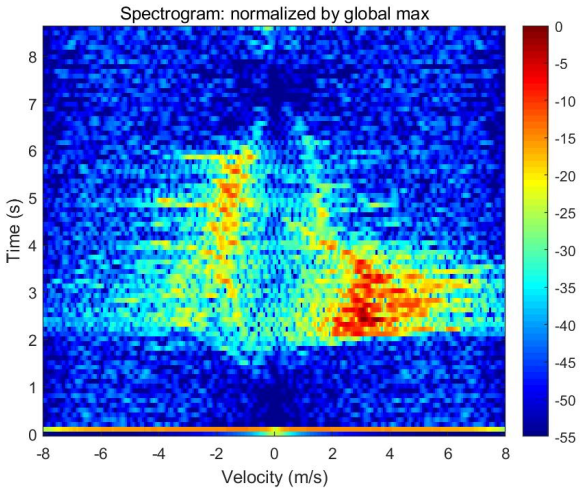


(a) Velocity vs time spectrogram

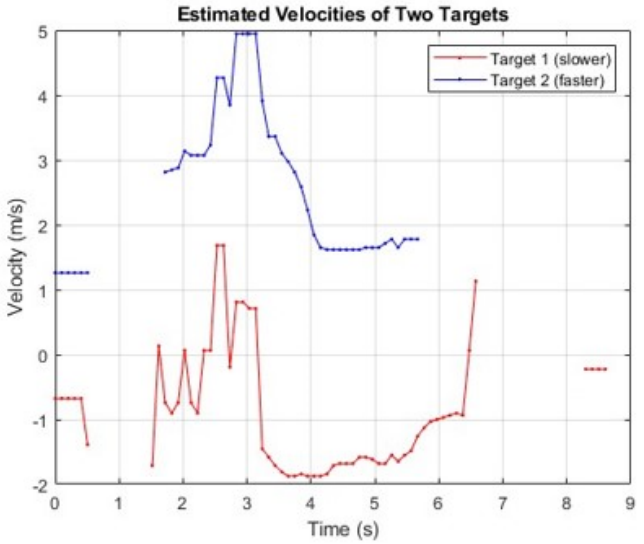


(b) Velocity vs time plot

Figure 25: CW IQ mode - Single target

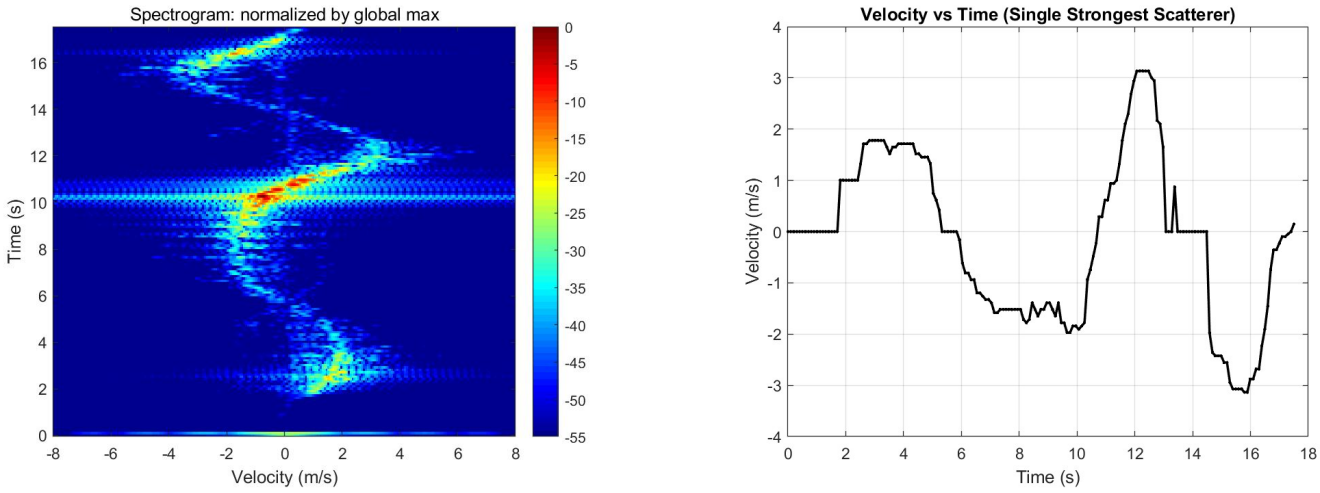


(a) Velocity vs time spectrogram



(b) Velocity vs time plot

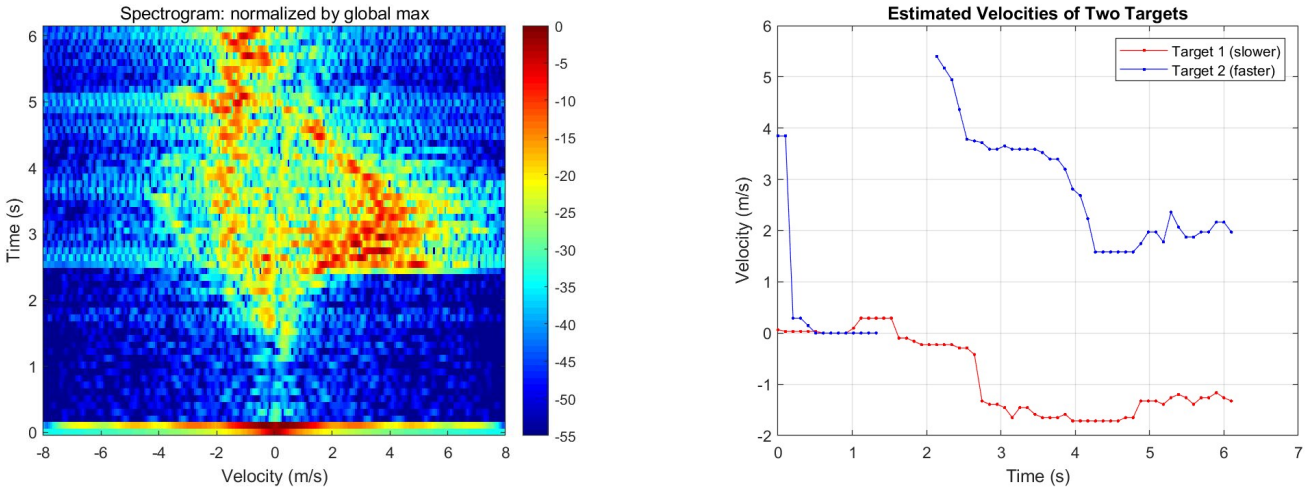
Figure 26: CW IQ mode - Multiple targets



(a) Velocity vs time spectrogram

(b) Velocity vs time plot

Figure 27: Low IF CW IQ mode - Single target



(a) Velocity vs time spectrogram

(b) Velocity vs time plot

Figure 28: Low IF CW IQ mode - Multiple targets

For single-target measurements, one subject walked away slowly from the radar and returned, then ran away fast and returned. For multi-target measurements, one subject ran away fast from the radar while another walked slowly toward it simultaneously. From the plots, we see the radar correctly captures their velocity (including velocity direction).

3.3 Effect of Power Amplifier on Range Performance

To evaluate the maximum detectable velocity range in continuous-wave (CW) operation, measurements were performed both with and without the external power amplifier connected to the SDR's transmitter output. The setup was kept identical for each transmit gain setting listed in Tables 1 and 2. The setup for carrying out the experiments includes placing the SDR system at a fixed location, and a target moving at approximately 2m/s away from the radar. The distance at which the target had disappeared from the spectrogram was recorded as the effective detection range. This process was repeated for each gain value to ensure consistency across measurements.
In every case, the setup was tested with and without the DC blocker in the GNU Radio schematic while processing the signals. A single schematic is used for measurements with a DC blocker and for measurements without a DC blocker; the schematic is shown in Figure 29.

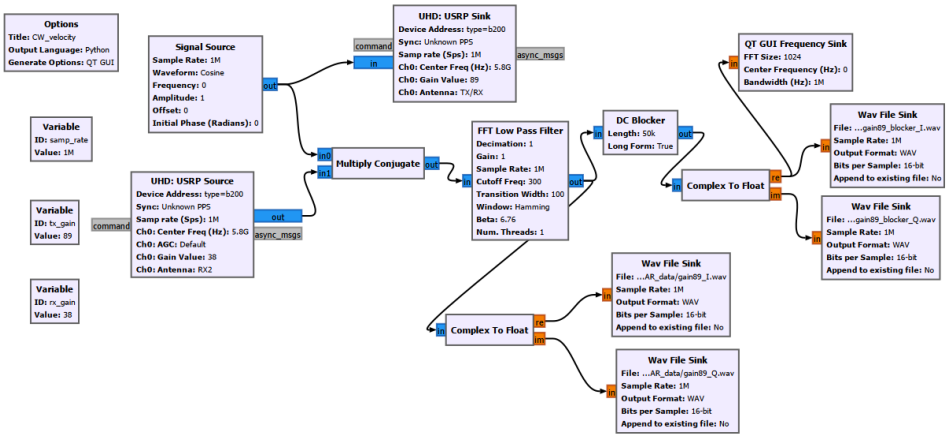


Figure 29: GNU Radio flowgraph for range evaluation

Upon examining all the results, we can observe that when the DC blocker was not used, a strong spectral line appeared around zero velocity in the Doppler spectrograms, indicating local oscillator (LO) leakage or insufficient transmit-receive (Tx/Rx) isolation. The inclusion of the DC blocker slightly improved detection performance by reducing this DC component.
From the tabulated results in 1 and 2, it can be seen that increasing the transmit gain and using the power amplifier both led to an increase in the effective detection range. In particular, even at lower transmit gain settings, the addition of the power amplifier noticeably improved the range performance, confirming that output power is a key factor. However, after an optimal point, i.e, at very high gain settings, the target signal disappeared completely due to LO leakage from the transmitter to the receiver, which desensitizes the receiver. This suggests that LO leakage and switch isolation are the most important factors to the overall limitation when the transmit gain is too high. The resulting spectrograms are included in the appendix (Fig. 33 - Fig.42).

Tx Gain at 5.8 GHz	Effective range (without DC Blocker)	Effective range (with DC Blocker)
50 dB	6 m	6 m
60 dB	14 m	14 m
70 dB	38 m	38 m
80 dB	44 m	44 m
89 dB	40 m	40 m

Table 1: Range performance without power amplifier

Tx Gain at 5.8 GHz	Effective range (without DC Blocker)	Effective range (with DC Blocker)
50 dB	14 m∼34 m	14 m∼34 m
60 dB	36 m	36 m
70 dB	46 m	72 m
80 dB	16 m	48 m
89 dB	<2 m	40 m∼50 m

Table 2: Range performance with power amplifier

3.4 Physiological Monitoring

In this experiment, the breathing pattern and heart rate of an individual were monitored using both Low-IF CW and CW radar modes. In the Low-IF configuration, a clear breathing pattern can be observed in the time-domain range variation as shown in Figure 30. By looking at Figure 32,we can see that the FFT of the range signal exhibits a distinct peak around 0.15 Hz, which corresponds to a breathing rate of approximately 9 breaths/min. Ideally, a secondary peak around 1 Hz (corresponding to a heart rate of about 60 bpm) should also appear; however, due to the small amplitude variation of chest wall motion from the heartbeat and also limitations of the setup such as noise floor and phase sensitivity, it is not clearly visible in the measured data.
In contrast, when operating in the standard CW mode, no clear breathing pattern can be identified, as we can see from Figure 31. This is because in CW mode, the radar operates near DC, where the local oscillator (LO) contributes significant phase and amplitude noise. As a result, slow-moving targets like breathing (velocity ~ 0.1 m/s) are masked by this low-frequency noise. Hence, the Low-IF CW mode is preferable for vital sign monitoring, as it effectively shifts the signal of interest away from the noisy DC region, allowing for improved detection of slow motions.

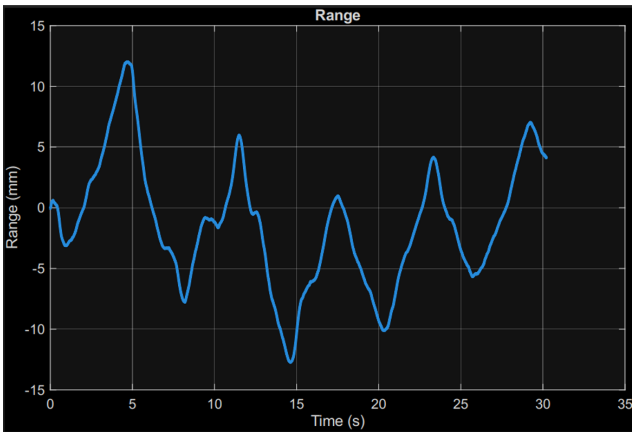


Figure 30: Range variation of breathing in Low-IF CW mode

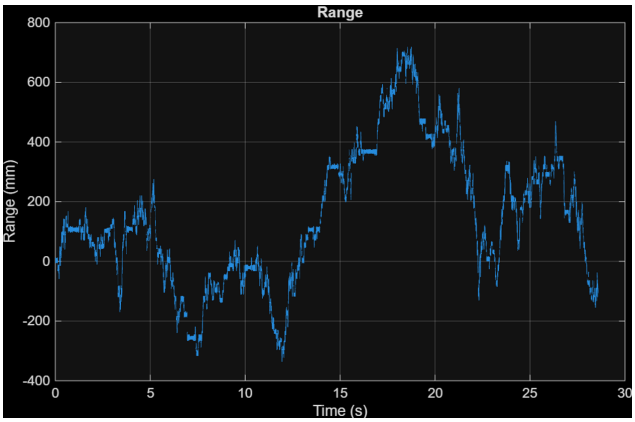


Figure 31: Range variation of breathing in normal CW mode

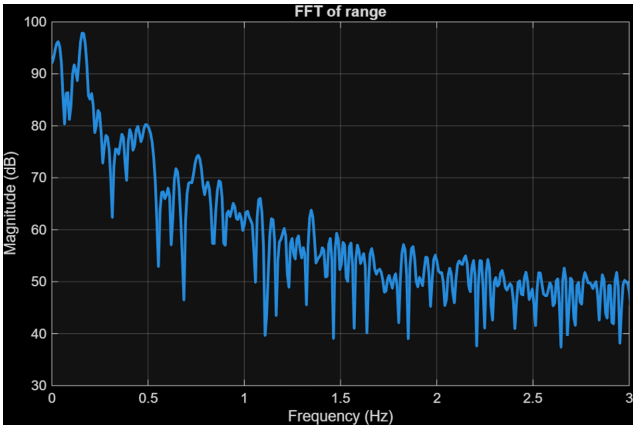


Figure 32: FFT of the range variation of breathing in Low-IF CW mode

4 Conclusion

At the beginning of the course, our main expectation was to gain practical, hands-on experience with radar hardware in addition to the theoretical knowledge acquired during lectures. This expectation was successfully met. Working directly with CW and FMCW radar systems by conducting measurements, processing the collected data in real time, and implementing relevant algorithms, provided us with valuable insights into the practical aspects of radar operation and signal processing. Performing experiments like multiple target range/velocity measurements also encouraged us to think about the system's limitations, for instance, when targets were lost or when the resolution was insufficient. This allowed us to connect theoretical concepts with practical challenges and to consider different experimental setups to better test the radar performance. Furthermore, we gained experience working with GNU Radio and interpreting schematic representations of the radar systems, which contributed to a deeper appreciation of the system architecture and signal flow. Overall, the course offered valuable hands-on exposure and deepened our conceptual understanding of radar systems.

Appendix

Plots

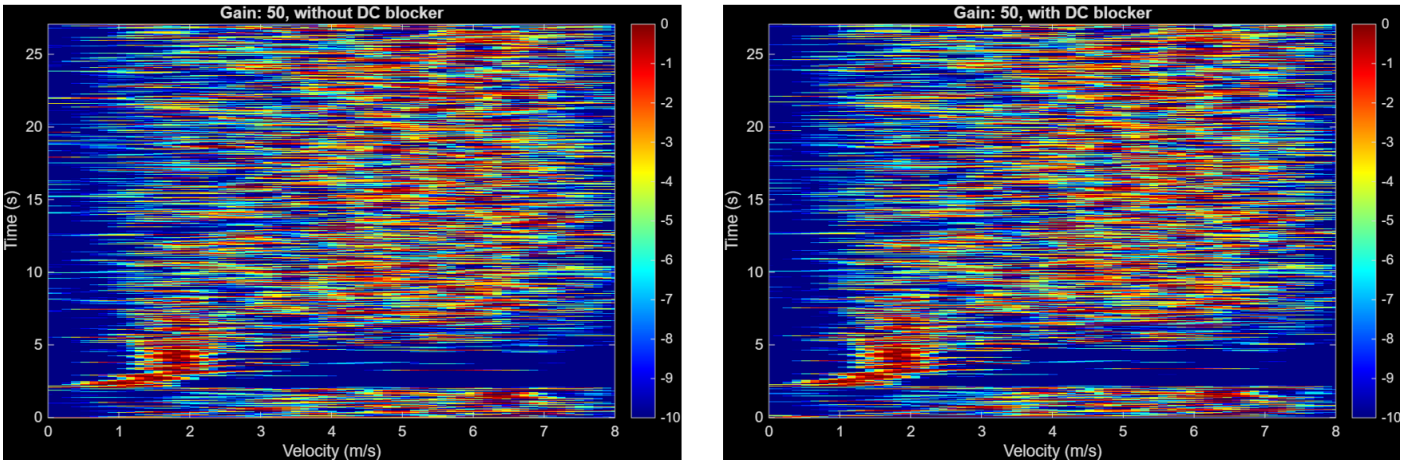


Figure 33: No power amplifier, Tx gain 50 dB

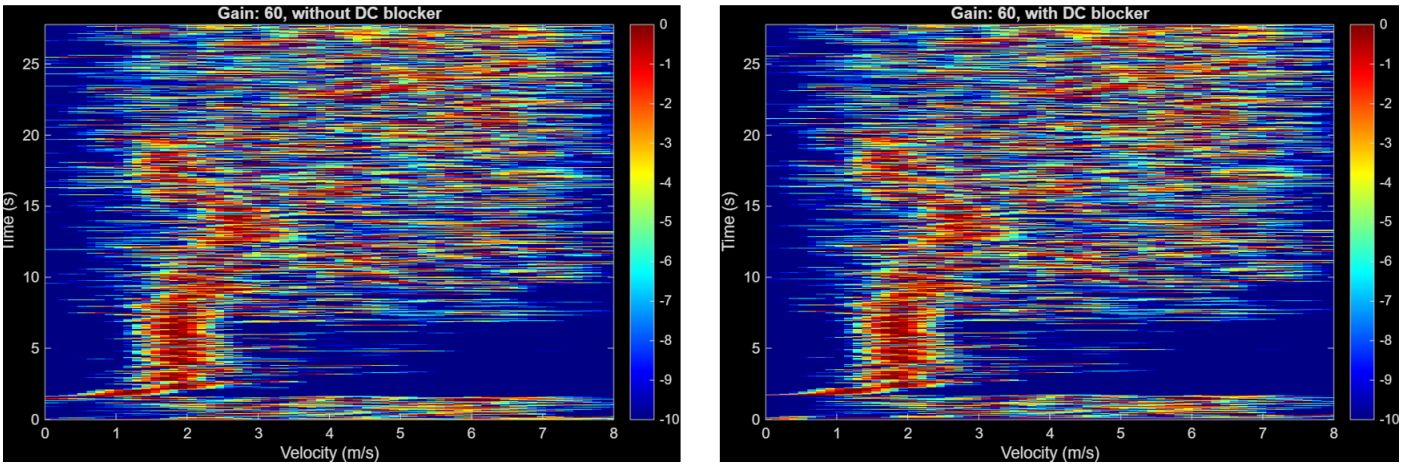
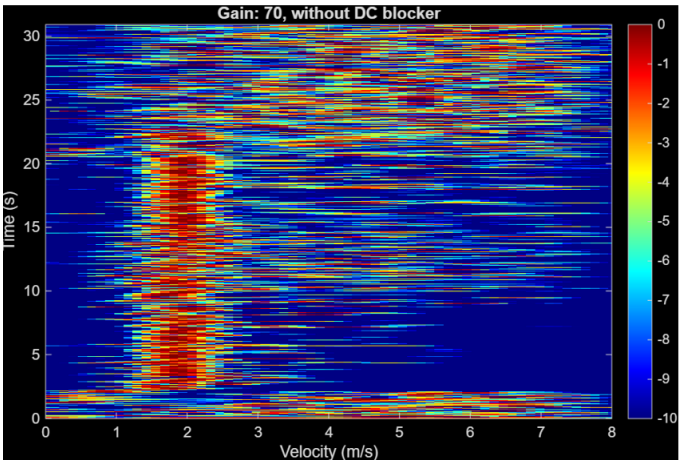


Figure 34: No power amplifier, Tx gain 60 dB



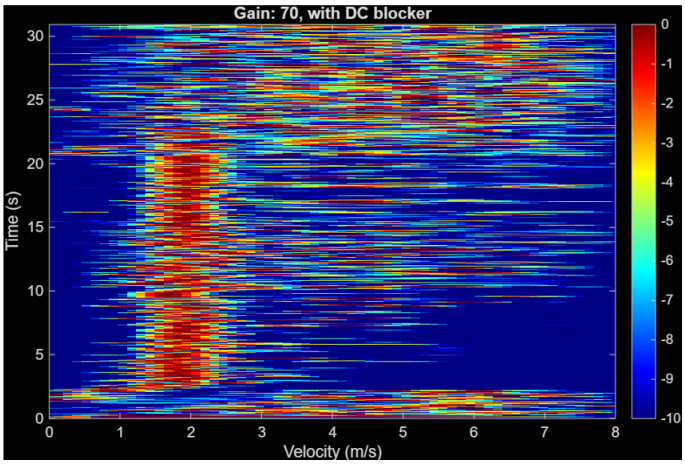
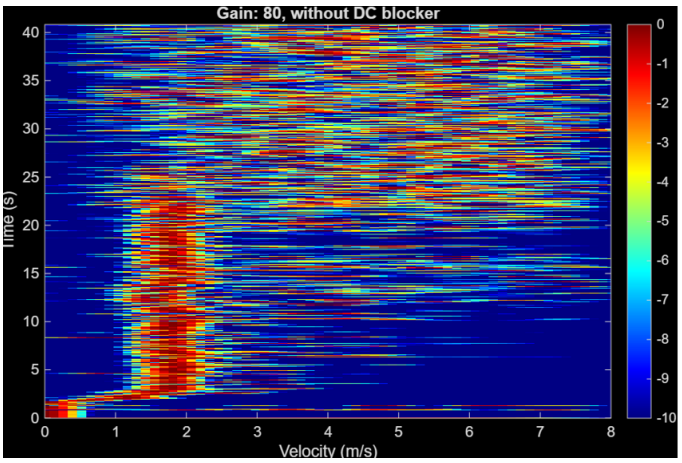


Figure 35: No power amplifier, Tx gain 70 dB



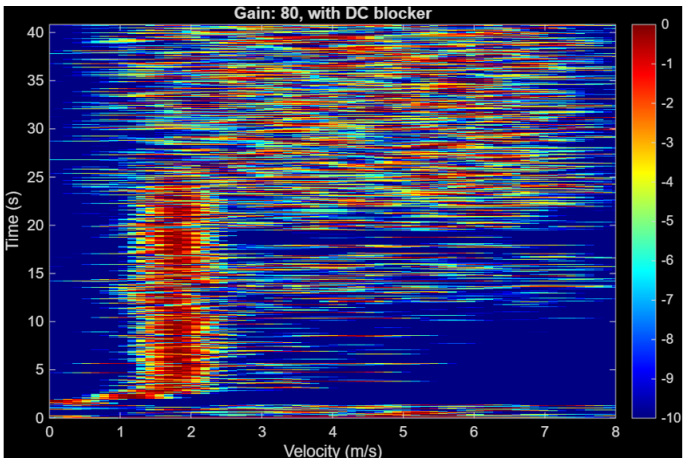
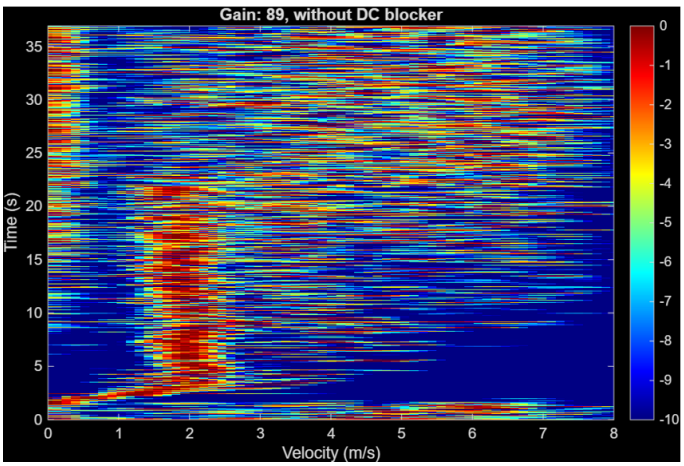


Figure 36: No power amplifier, Tx gain 80 dB



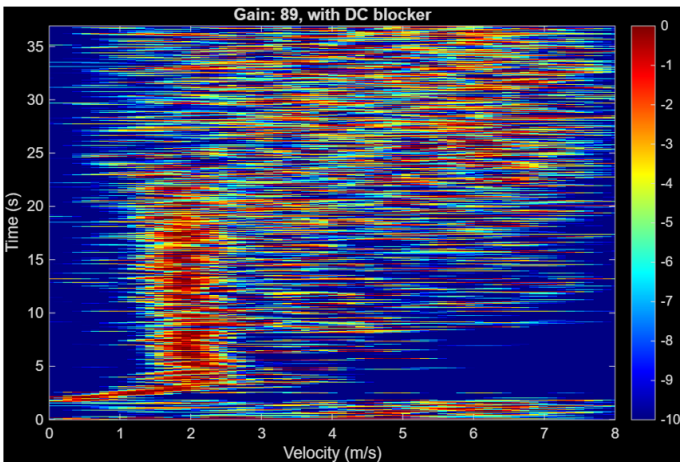
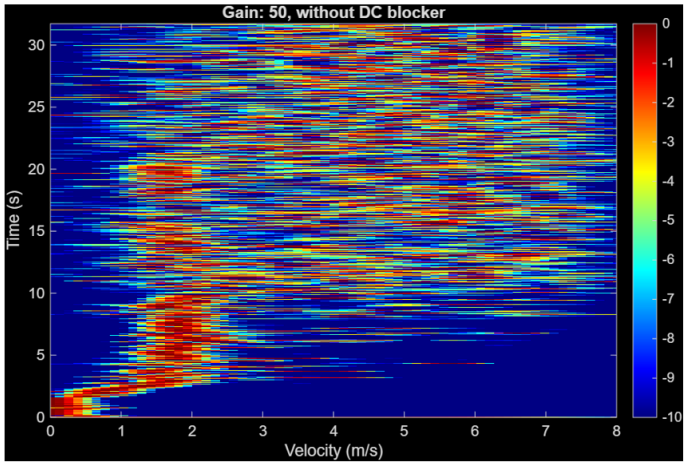


Figure 37: No power amplifier, Tx gain 89 dB



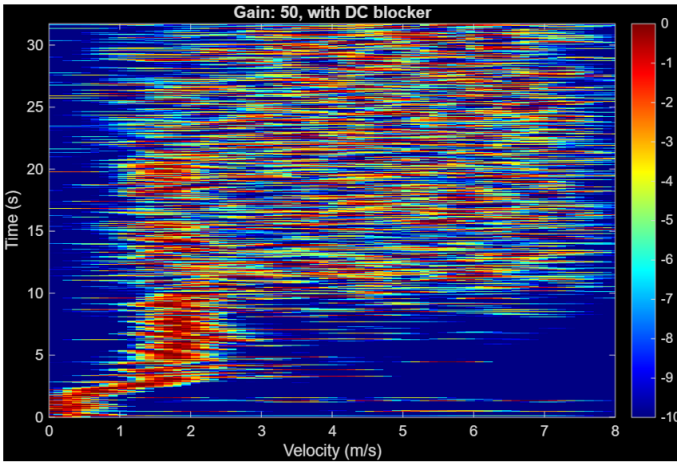
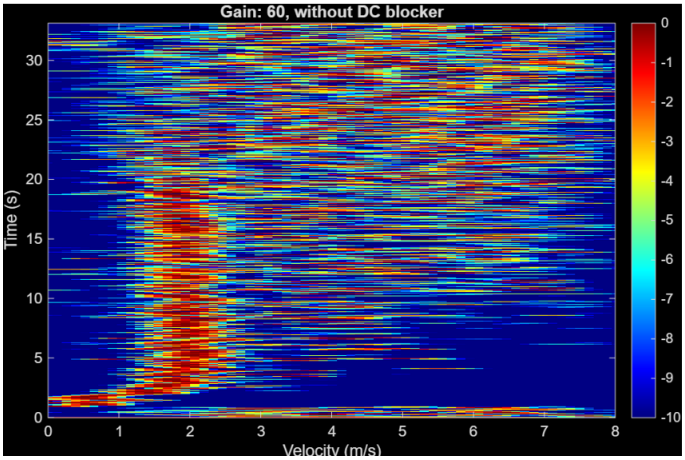


Figure 38: With power amplifier, Tx gain 50 dB



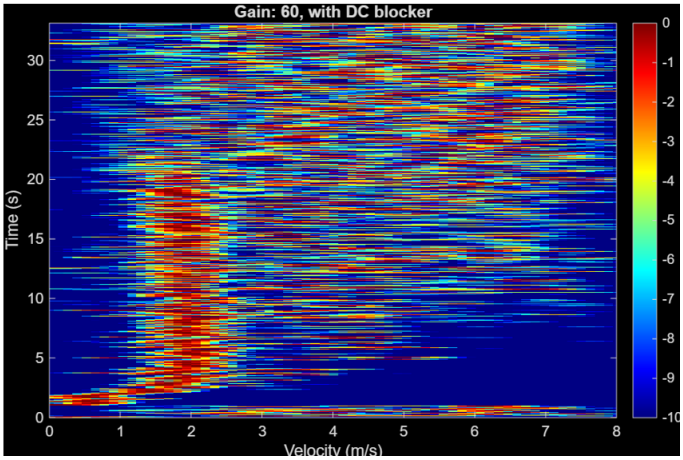
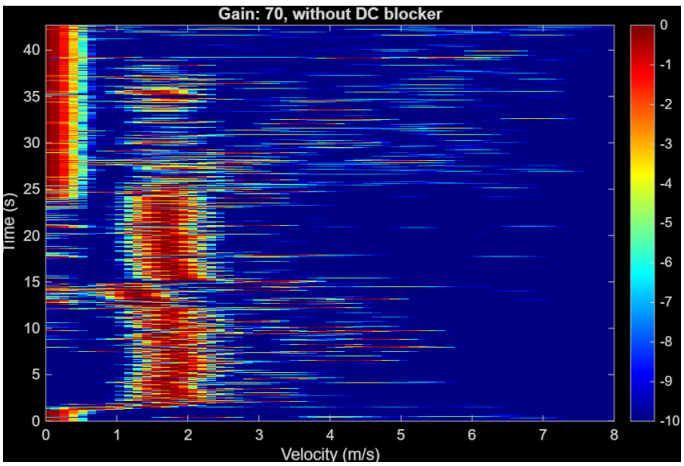


Figure 39: With power amplifier, Tx gain 60 dB



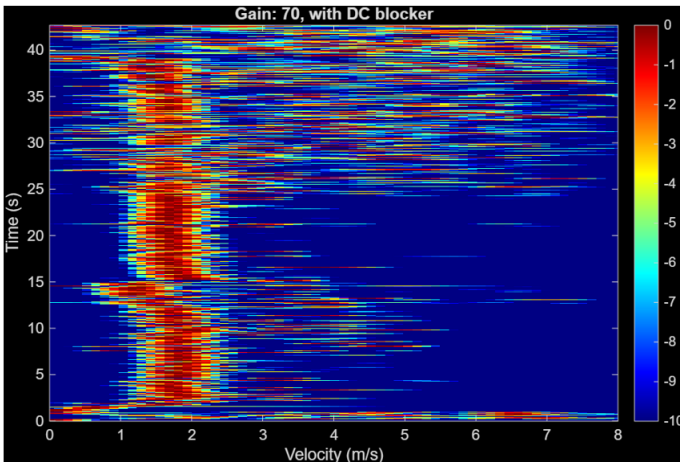


Figure 40: With power amplifier, Tx gain 70 dB

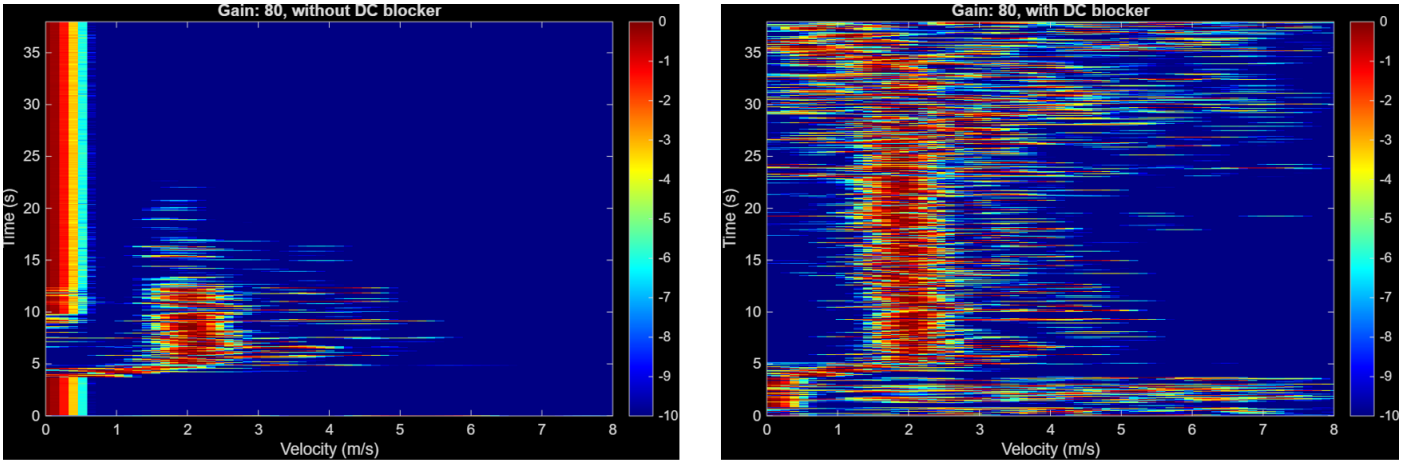


Figure 41: With power amplifier, Tx gain 80 dB

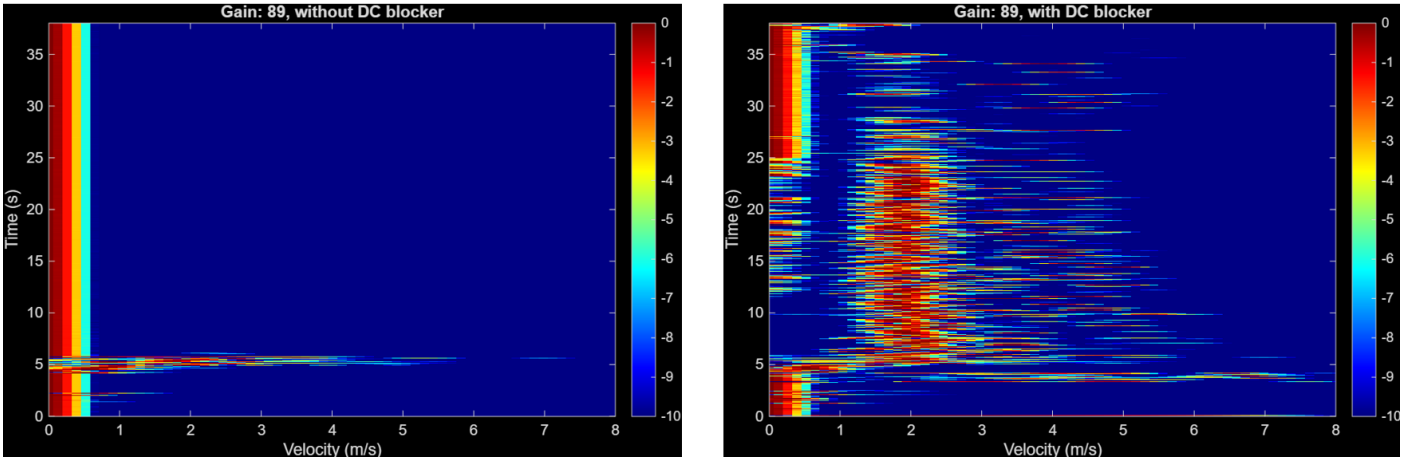


Figure 42: With power amplifier, Tx gain 89 dB

Code

Listing 1: Code for velocity measurements (CW)

```
clc; clear all; close all;

%% Step 0: Read audio file
use_IQ = false;

for gain = [60, 70]
    for use_blocker = [true, false]
        name =  With_PA/PA_gain  + gain;
        if use_blocker
            I_name = name +  _blocker_I.wav ;
            Q_name = name +  _blocker_Q.wav ;
        else
            I_name = name +  _I.wav ;
            Q_name = name +  _Q.wav ;
        end

        if use_IQ
            [I_data, Fs] = audioread(I_name);
            [Q_data, Fs] = audioread(Q_name);
        else
```

```
    disp(I_name)
    [data, Fs] = audioread(I_name);
end

%% Step 1: Inspect both channels
% figure; hold on;
% plot(I_data(:,1),'b');    % channel 1 (inverted for
   visibility)
% plot(Q_data(:,1),'r');     % channel 2
% xlabel('Sample number');
% ylabel('Amplitude');
% legend('Channel 1','Channel 2');
% title('Comparison of both channels');

%% Pick radar channel
if use_IQ
    radar = complex(I_data(:,1), Q_data(:,1));
else
    radar = data(:,1);          % Channel 1 contains the radar
        data
end
t = (0:length(radar)-1)/Fs;

%% Split radar data into sweeps
Tp = 0.05;                     % in seconds - pulse length
N = round(Tp * Fs);           % samples per sweep
M = floor(length(radar)/N)-1;  % number of complete sweeps

radar = radar(1:M*N);  % truncate radar to integer number of
   sweeps
sweeps = reshape(radar, [N, M]).';   % size M x N

fprintf('Matrix size: %d sweeps x %d samples\n', M, N);

% %% Optional: plot first sweep
% t_sweep = (0:N-1)/Fs;
% figure;
% plot(t_sweep, sweeps(1,:));
% xlabel('Time (s)');
% ylabel('Amplitude');
% title('First radar sweep');

%% MS clutter rejection
disp(mean(sweeps(:)))
mean_val = mean(sweeps(:));   % global mean
sweeps_ms = sweeps - mean_val;

w = hann(N).';
sweeps_ms = sweeps_ms .* w;

% % Optional, just for visualization - Plot first sweep after
   MS
% figure;
```

```
% plot(t_sweep, sweeps_ms(1,:));
% xlabel('Time (s)');
% ylabel('Amplitude');
% title('First sweep after MS clutter rejection (global mean)
   ');

%% FFT with zero-padding and log scale
Y = fft(sweeps_ms, 4*N, 2);        % FFT across columns, zero-
   padding to 4*N
if use_IQ
    Y = fftshift(Y, 2);
end
Y_dB = 20*log10(abs(Y));           % conversion to dB

if ~use_IQ
    Y_dB_half = Y_dB(:, 1:2*N); % Keep up to Fs/2
else
    Y_dB_half = Y_dB;
end

%% Maximum representable frequency
Fmax = Fs/2;  % Nyquist

%% Normalizations

Y_norm1 = Y_dB_half - max(max(Y_dB_half)); % subtract maximum
   of entire lower-half matrix

Y_norm2 = Y_dB_half - max(Y_dB_half,[],2);  % row-wise max
   subtraction

%% Doppler frequency array to velocity array
if use_IQ
    f_axis = linspace(-Fmax, Fmax, 4*N);
else
    f_axis = linspace(0, Fmax, 2*N);
end

% CW radar conversion: v = c/(2*fc) * fD
c = 3e8;
fc = 5.8e9;
v_axis = (c/(2*fc)) * f_axis;

%% Time axis for sweeps
t_axis = linspace(0, Tp*M, M);

%% Step 9: Plot spectrograms

% figure;
% imagesc(v_axis, t_axis, Y_norm1);
% set(gca, 'YDir', 'reverse');
% axis xy;
% xlabel('Velocity (m/s)');
```

```
        % ylabel('Time (s)');
        % title('Spectrogram: normalized by global max');
        % colorbar;
        % caxis([-45 0]);
        % colormap(jet);
        % if use_IQ
        %     xlim([-8 8]);
        % else
        %     xlim([0 8]);
        % end

        figure;
        imagesc(v_axis, t_axis, Y_norm2);
        set(gca, 'YDir', 'reverse');
        axis xy;
        xlabel('Velocity (m/s)');
        ylabel('Time (s)');
        title_name =  Gain:   + gain;
        if use_blocker
            title_name = title_name +  , with DC blocker ;
        else
            title_name = title_name +  , without DC blocker ;
        end
        title(title_name);
        colorbar;
        caxis([-10 0]);   % set color scale from -55 dB (blue) to 0 dB
            (yellow/red)
        colormap(jet);
        if use_IQ
            xlim([-8 8]);
        else
            xlim([0 8]);
        end
        drawnow;
    end
end
```

Listing 2: Code for range measurements (FMCW)

```
clear all; close all; clc
% Read the file
[data, Fs] = audioread( fmcw_single.wav );

% Make time axis for plotting the waveforms
t = (0:size(data,1)-1)/Fs;

% Separate the data and Visualise
sync_data = data(:, 2);
back_scatter_data = data(:, 1);
%plot(t, sync_data)
%figure
%plot(t, back_scatter_data)
%xlabel('Time (s)'), ylabel('Amplitude'), title('Mono Waveform')
```

```
% Split the Sync data
threshold = 0;
sync_data = sync_data > threshold;

% define up chirp duration and samples
Tp = 0.005;
chirp_duration = round(Tp*Fs);

% find the data where sync pulse is one
combined_data = sync_data.*back_scatter_data;

% find the segments in the data where sync pulse is one and
   concatenate them into a matrix
d = diff([0 transpose(sync_data) 0]);
starts = find(d == 1);
ends   = find(d == -1) - 1;
num_segments = length(starts);
seg_lengths = ends - starts + 1;
max_length = max(seg_lengths);
segments = zeros(num_segments, max_length);

for k = 1:num_segments
    seg = combined_data(starts(k): min(ends(k), starts(k) +
       chirp_duration+1));
    segments(k, 1:length(seg)) = seg;
end

% MS clutter rejection
col_mean = mean(segments, 1);
segments = segments - col_mean;

% generating the time axis
time = starts/Fs;

% performing MTI
data_2MTI = segments(2:size(segments,1),:) - segments(1:size(segments
   ,1)-1,:);
data_3MTI = segments(3:size(segments,1),:) - 2*segments(2:size(
   segments,1)-1,:) ...
    + segments(1:size(segments,1)-2,:);

% Performing ifft
N = chirp_duration;
Y1= abs(ifft(segments, 4*N, 2));
Y2= abs(ifft(data_2MTI, 4*N, 2));
Y3= abs(ifft(data_3MTI, 4*N, 2));

% converting to db scale
Y1_dB = 20 * log10(Y1);
Y2_dB = 20 * log10(Y2);
Y3_dB = 20 * log10(Y3);

% taking the lower half of the matrix
```

```
half_size = floor(size(Y1_dB, 2)/2);
Y1_dB_lower = Y1_dB(:, 1:half_size);
Y2_dB_lower = Y2_dB(:, 1:half_size);
Y3_dB_lower = Y3_dB(:, 1:half_size);

% Y1_dB_lower = Y1_dB(:, 1:12*N+1);
% Y2_dB_lower = Y2_dB(:, 1:12*N+1);
% Y3_dB_lower = Y3_dB(:, 1:12*N+1);

% Normalisation
Y1_norm = Y1_dB_lower - max(max(Y1_dB_lower));
Y2_norm = Y2_dB_lower - max(max(Y2_dB_lower));
Y3_norm = Y3_dB_lower - max(max(Y3_dB_lower));

% compute Rmax
c = 299792458;
f_start = 2.405 * 10^9;
f_stop = 2.495 * 10^9;
delta_f = f_stop - f_start;
delta_R = c/(2*delta_f);
R_max = (N*delta_R)/2;
y =  linspace(0, R_max, half_size);

%plot the spectograms
imagesc(y, time, Y1_norm, [-50, 0])
title( Range with MS and no MTI )
xlabel( Range (m) )
ylabel( Time (s) )
xlim([0, 30])
colorbar
figure

time_2MTI = time(1:end-1);
imagesc(y, time_2MTI, Y2_norm, [-50, 0])
title( Range with MS and 2-pulse MTI )
xlabel( Range (m) )
ylabel( Time (s) )
xlim([0, 30])
colorbar
figure


time_3MTI = time(1:end-2);
imagesc(y, time_3MTI, Y3_norm, [-50, 0])
title( Range with MS and 3-pulse MTI )
xlabel( Range (m) )
ylabel( Time (s) )
xlim([0, 30])
colorbar

%%
min_dist = delta_R;      % Minimum spacing between peaks (m)
min_height = -25;   % Power threshold in dB
```

```
chosen_Y = Y3_norm;
chosen_time = time_3MTI;

r_est1 = zeros(size(chosen_Y,1),1); % Preallocate
r_est2 = zeros(size(chosen_Y,1),1);

for i = 1:size(chosen_Y,1)
    row = chosen_Y(i,:);
    [peakVals, locs] = findpeaks(row, y, ...
        'MinPeakHeight', min_height, ...
        'MinPeakDistance', min_dist);
    if isempty(peakVals)
        if i > 1
            r_est1(i) = r_est1(i-1);
            r_est2(i) = r_est2(i-1);
        else
            continue;
        end

    elseif isscalar(peakVals)
        r_est1(i) = locs(1);
        r_est2(i) = NaN;
    else
        % Take two strongest peaks
        [~, sortIdx] = maxk(peakVals, 2);
        peakLocs = locs(sortIdx);

        % Assign consistently: smaller range = Target 1, larger =
           Target 2
        r_sorted = sort(peakLocs);
        r_est1(i) = r_sorted(1);
        r_est2(i) = r_sorted(2);
    end
end
%%
figure;
plot(chosen_time, r_est1);
xlabel('Time (s)'); ylabel('Range (m)');
title('Range vs Time (Single Strongest Scatterer)');
xlim([0, 25]);
grid on;


figure;
plot(chosen_time, r_est2);
xlabel('Time (s)'); ylabel('Range (m)');
title('Range vs Time (Second Strongest Scatterer)');
xlim([0, 25]);
grid on;

figure;
plot(chosen_time, r_est1, 'DisplayName','Target 1 (close)'); hold on;
```

```
plot(chosen_time, r_est2, 'DisplayName','Target 2 (farther)');
xlabel('Time (s)');
ylabel('Range (m)');
title('Estimated Range of Two Targets');
legend show; grid on;

%%
half_size = round(size(Y3, 2)/2);
Y3_lower = Y3(:, 1:half_size);

% Plots without noise reduction
[val, idx] = max(Y3_lower, [], 2);
strongest_range = y(idx);
figure
plot(time_3MTI, val)
title( Size of scatter )
figure
plot(time_3MTI, strongest_range)
title( FMCW, 3MTI, 4N padding, MS clutter rejection )
legend('Strongest scatter')
xlabel('Time (s)')
ylabel('Range (m)')
%%

detection_threshold = 0.01;
max_vel = 5;

range = zeros(size(Y3_lower,1),1);
velocity = zeros(size(Y3_lower,1),1);

target_detected = false;
latest_update = 0;
for i = 1:size(Y3_lower, 1)
    row = Y3_lower(i, :);
    if target_detected
        reasonable_vel = false;
        [val, idx] = max(row);
        delta_length = y(idx) - range(latest_update);
        delta_time = time_3MTI(i) - time_3MTI(latest_update);
        vel = delta_length / delta_time;
        %disp(vel)
        if abs(vel) < max_vel % | i >= size(Y3_lower, 1)
            for j = latest_update:i
                velocity(j) = vel;
                range(j) = range(latest_update) + ...
                    vel * (time_3MTI(j) - time_3MTI(latest_update));
            end
            latest_update = i;
        end

    else
        if max(row) > detection_threshold
            target_detected = true;
```

```
            [val, idx] = max(row);
            range(i) = y(idx);
            latest_update = i;
            first_target = i;
        end
    end
end
range(1:first_target-1) = [];
velocity(1:first_target-1) = [];
hold on
plot(time_3MTI, range)
legend('Strongest scatter', 'Noise reduced target')
xlabel('Time (s)')
ylabel('Range (m)')
%%
figure
plot(time_3MTI, velocity)
hold on
velocity_smooth = movmean(velocity, 500);
plot(time_3MTI, velocity_smooth)
title( FMCW, 3MTI, 4N padding, MS clutter rejection )
xlabel('Time (s)')
ylabel('Velocity (m/s)')
legend('Noise reduced velocity', 'Noise reduced velocity moving
   average')
```

Listing 3: Code for Realtime velocity measurements

```
clc; clear; close all;

%% Parameters
Fs = 44100;                % Sampling frequency (match radar output!)
nBits = 16;                % Bits per sample
nChannels = 2;             % Stereo input (radar on channel 1)
Tp = 0.5;                  % Sweep time [s]
N = round(Tp*Fs);          % Samples per sweep
c = 3e8;                   % Speed of light
fc = 2.445e9;              % Radar carrier frequency

%%
audiodevinfo
%% Create recorder object
recObj = audiorecorder(Fs, 16, 1, DeviceID);
disp('>> Starting real-time radar acquisition...');
record(recObj);            % start recording in background

%% Velocity axis
f_axis = linspace(0, Fs/2, 2*N);
v_axis = (c/(2*fc)) * f_axis;

%% Initialize spectrogram buffer
maxSweeps = 200;   % how many sweeps to display in the live
   spectrogram
Y_buffer = nan(maxSweeps, 2*N);  % store FFT magnitudes
```

```
t_axis = (0:maxSweeps-1) * Tp;   % time axis

%% Prepare plot
figure;
hImg = imagesc(v_axis, t_axis, Y_buffer, [-55 0]); % dB range
set(gca, 'YDir', 'normal');
xlabel('Velocity (m/s)');
ylabel('Time (s)');
title('Real-time Spectrogram');
colormap(jet);
colorbar;
ylim([0 maxSweeps*Tp]);
xlim([0 30]);

%% Real-time loop
sweepCount = 0;
while isrecording(recObj)
    pause(Tp);   % wait one sweep duration

    % Fetch all recorded samples so far
    data = getaudiodata(recObj);
    radar = data(:,1);   % use first channel

    if length(radar) >= N
        % Take only the last sweep
        sweep = radar(end-N+1:end);

        % Mean subtraction
        sweep_ms = sweep - mean(sweep);

        % FFT
        Y = fft(sweep_ms, 4*N);
        Y_dB = 20*log10(abs(Y(1:2*N)));
        Y_dB = Y_dB - max(Y_dB); % normalize per sweep

        % Update spectrogram buffer
        sweepCount = sweepCount + 1;
        rowIdx = mod(sweepCount-1, maxSweeps) + 1; % circular buffer
        Y_buffer(rowIdx,:) = Y_dB;

        % Update plot
        set(hImg, 'CData', Y_buffer, ...
                  'YData', (0:maxSweeps-1)*Tp, ...
                  'XData', v_axis);
        ylim([0 maxSweeps*Tp]); % sliding time window
        drawnow;
    end
end
```

Listing 4: Code for Realtime Range measurements

```
Fs = 44100;                % Adjust to your soundcard sampling rate
Tp = 0.005;                % Chirp duration (s)
chirp_samples = round(Tp * Fs);
```

```
%Run this command to see which audio devices are connected to see the
   USB sound card
%audiodevinfo

% Setup audio recorder (stereo: ch1=backscatter, ch2=sync)
deviceID = 1;  % (example, replace with your USB sound cards ID
   from step 1)
recObj = audiorecorder(Fs, 16, 2, deviceID);
record(recObj);

% Radar parameters
c = 299792458;
f_start = 2.405e9;
f_stop  = 2.495e9;
delta_f = f_stop - f_start;
delta_R = c/(2*delta_f);
R_max   = (chirp_samples * delta_R)/2;
y = linspace(0, R_max, 2*chirp_samples);

% Spectrogram buffer
max_frames = 200; % number of chirps shown in plot
spec_buffer = -50*ones(max_frames, 2*chirp_samples); % initialize with
    -50 dB

figure;
h = imagesc(y, (0:max_frames-1)*Tp, spec_buffer, [-50, 0]);
xlabel('Range (m)');
ylabel('Time (s)');
title('Real-time FMCW Range Spectrogram (2-pulse MTI)');
colorbar;
xlim([0,100]);

frame_idx = 0;
prev_chirp = []; % store previous chirp for MTI

%% Continuous processing loop
while true
    pause(Tp);  % wait approx. one chirp

    % Get available audio
    data = getaudiodata(recObj);
    if size(data,1) < chirp_samples
        continue; % not enough samples yet
    end

    % Separate channels
    backscatter = data(:,1);
    sync_data   = data(:,2) > 0; % threshold

    % Detect chirp start/stop using sync
    d = diff([0; sync_data; 0]);
    starts = find(d == 1);
```

```
    ends   = find(d == -1) - 1;

    if isempty(starts)
        continue; % no chirp detected yet
    end

    % Process the last complete chirp
    k = starts(end);
    if k+chirp_samples-1 > length(backscatter)
        continue; % wait for full chirp
    end
    chirp_segment = backscatter(k:k+chirp_samples-1);

    % Mean subtraction (MS clutter rejection)
    chirp_segment = chirp_segment - mean(chirp_segment);

    % Apply 2-pulse MTI if previous chirp exists
    if ~isempty(prev_chirp)
        mti_chirp = chirp_segment - prev_chirp;

        % FFT and normalization
        Y = abs(ifft(mti_chirp, 4*chirp_samples));
        Y_dB = 20*log10(Y(1:2*chirp_samples+1));
        Y_norm = Y_dB - max(Y_dB);

        % Update buffer
        frame_idx = frame_idx + 1;
        if frame_idx > max_frames
            spec_buffer = circshift(spec_buffer, -1, 1);
            spec_buffer(end,:) = Y_norm;
        else
            spec_buffer(frame_idx,:) = Y_norm;
        end

        % Update spectrogram plot
        set(h, 'CData', spec_buffer);
        drawnow;
    end

    % Store current chirp as previous
    prev_chirp = chirp_segment;
end
```

Listing 5: Code for Range-doppler measurements (FMCW)

```
clear all; close all; clc
% 0. Draw an empty plot to pre-load the plot tool
imagesc([], [], [], [-50, 0]);
xlabel( Range (m) );
ylabel( Velocity (m/s) );
xlim([0, 100]);
colorbar;
drawnow;
```

```
% 1. Read audio file
[data, Fs] = audioread( fmcw_range_doppler.wav );

% 2. Separate sync data and back scatter data
sync_data = data(:, 2);
back_scatter_data = data(:, 1);
%plot(sync_data)
%figure
%plot(back_scatter_data)

% 3. Place everything coming up in a while loop
idx = 1;
max_idx = size(sync_data, 1);
chunk = round(1 * Fs);

strongest_range = [];
strongest_vel = [];
counter = 0;
while idx + chunk <= max_idx
    counter = counter + 1;
    chunked_sync_data = sync_data(idx: idx + chunk);
    chunked_back_scatter_data = back_scatter_data(idx: idx + chunk);

    % 4. Binarize sync data
    threshold = 0;
    chunked_sync_data = chunked_sync_data > threshold;

    % 4 (again). Split the back scatter data
    Tp = 0.005;
    chirp_duration = round(Tp*Fs);
    combined_data = chunked_sync_data.*chunked_back_scatter_data;
    d = diff([0 transpose(chunked_sync_data) 0]);
    starts = find(d == 1);
    ends   = find(d == -1) - 1;
    num_segments = length(starts);
    seg_lengths = ends - starts + 1;
    max_length = max(seg_lengths);
    segments = zeros(num_segments, max_length);

    for k = 1:num_segments
        seg = combined_data(starts(k): min(ends(k), starts(k) +
           chirp_duration));
        segments(k, 1:length(seg)) = seg;
    end

    % 5. MS clutter rejection
    col_mean = mean(segments, 1);
    segments = segments - col_mean;

    % 6. 2 pulse MTI
    data_2MTI = segments(2:size(segments,1),:) - segments(1:size(
       segments,1)-1,:);
    %data_2MTI = segments;
```

```
    % 7. IFFT, no zero padding
    %N = chirp_duration;
    %N = half_size*2;
    Y1 = ifft(data_2MTI, [], 2);
    Y2 = ifft(Y1, [], 1);

    Y2_dB = 20 * log10(abs(Y2));

    % 8. Keep lower part of matrix
    half_size = floor(size(Y2_dB, 2)/2);
    N = half_size*2;
    Y2_dB_lower = Y2_dB(:, 1:half_size);

    % 9. Normalization
    Y2_norm = Y2_dB_lower - max(max(Y2_dB_lower));

    % 11 (no 10). Create data for y-axis
    c = 299792458;
    f_start = 2.405 * 10^9;
    f_stop = 2.495 * 10^9;
    f_c = (f_start + f_stop) / 2;
    f_D = linspace(0, 1/(2*Tp), size(Y2_norm, 1));
    velocity_array = (c/(2*f_c)) * f_D;

    % 12. Create data for x-axis
    delta_f = f_stop - f_start;
    delta_R = c/(2*delta_f);
    R_max = (N*delta_R)/2;
    range_array = linspace(0, R_max, size(Y2_norm, 2));

    % 13. Generate final spectrogram
    imagesc(range_array, velocity_array, Y2_norm, [-50, 0]);
    xlabel( Range (m) );
    ylabel( Velocity (m/s) );
    xlim([0, 40]);
    colorbar;
    drawnow;

    %
    Y2_lower = abs(Y2(:, 1:half_size));
    [val, linearIndex] = max(Y2_lower(:));
    [row, col] = ind2sub(size(Y2_lower), linearIndex);
    strongest_range = [strongest_range, range_array(col)];
    strongest_vel = [strongest_vel, velocity_array(row)];


    % 14. Increment the index
    idx = idx + chunk;


end
time_axis = linspace(0, counter*chunk/Fs, counter);
```

```
figure
plot(time_axis, strongest_vel)
title( FMCW, 2MTI, no zero padding, MS clutter rejection )
legend('Strongest scatter')
xlabel('Time(s)')
ylabel('Velocity(m/s)')
figure
plot(time_axis, strongest_range)
title( FMCW, 2MTI, no zero padding, MS clutter rejection )
legend('Strongest scatter')
xlabel('Time(s)')
ylabel('Range(m)')
```

Listing 6: Code for Time Domain Correlation for simulated targets

```
% ================== SAR Time-Domain Correlation (Simulated Targets)
   ==================
clearvars; close all; clc;

% ---------- 0) Basic Parameters (modify for tasks  a d ) ----------
Fs    = 50e3;
Tp    = 5e-3;                   % Up-chirp duration
f1    = 2.4e9; f2 = 2.5e9;      % Frequency band (use 5.4  5  .5 GHz for
   part d)
BW    = f2 - f1;
fc    = (f1 + f2)/2;
c     = 3e8;
N     = round(Fs*Tp);           % Samples per chirp (N = Fs*Tp)

lambda  = c/fc;
spacing = lambda/2;

% Rail positions (meters)
xp = -3.000 : spacing : 3.000;   % cross-range radar track
yp = 0;

% Target positions (meters)
x_pos = [-2.050  0.000  2.500];
y_pos = [0 0 0];
z_pos = [10 20 30];

% Time array (row vector)
t = linspace(0, Tp, N);          % 1 x N

% ---------- 1) Generate Simulated Data using function ----------
% Expect SAR_Data to be: numPositions x 1 x N (or numPositions x
   channels x N)
SAR_Data = SAR_Data_Generation(x_pos, y_pos, z_pos, xp, yp, N, t, fc,
   BW, Tp);

% ---------- 2) Define z-axis (down-range) ----------
dR    = c/(2*BW);                % Range resolution  R
Rmax = N * dR / 2;               % Maximum unambiguous range
z_range = linspace(0, Rmax, N);  % 1 x N
```

```
% ---------- 3) Define x-axis (cross-range) ----------
x_range = xp;                       % 1 x M
M = numel(xp);

% ---------- 4) Initialize Image Matrix ----------
img = zeros(numel(x_range), numel(z_range)); % M x N

% ---------- 5) Prepare measurement matrix (M x N) ----------
% Adjust extraction depending on the exact SAR_Data shape:
% If SAR_Data is (M x 1 x N) then squeeze gives M x N:
meas = squeeze(SAR_Data(:,1,:));     % should give M x N
% If your SAR_Data is M x N directly, use meas = SAR_Data;

% Ensure meas orientation: rows = radar positions, cols = time samples
if size(meas,1) ~= M || size(meas,2) ~= N
    error('Unexpected meas size: expected [%d x %d], got [%d x %d].',
       M, N, size(meas,1), size(meas,2));
end

% Convert t to row vector (ensure 1 x N) and meas is M x N
t = reshape(t, 1, []);   % 1 x N

% ---------- 6) Time-Domain Correlation (pixel-wise focusing)
   ----------
tic;
for ix = 1:M
    x_img = x_range(ix);                    % scalar
    % cross-range distance vector from pixel to all radar pos (1 x M)
    dx = x_img - xp;                        % 1 x M
    % absolute not necessary inside sqrt, but keep sign-consistent:
    r_x = abs(dx);                          % 1 x M

    for iz = 1:N
        z_img = z_range(iz);                % scalar
        % slant range vector (1 x M)
        r = sqrt(r_x.^2 + z_img^2);       % 1 x M
        % two-way delays as column vector (M x 1)
        td = (2 * r / c).';               % make it M x 1 (transpose)

        % Build phase kernel (M x N) using implicit expansion:
        % phase(m,n) = exp(j*2*pi*(fc*td(m) + (BW/Tp)*td(m) * t(n)))
        phase = exp(1j * 2 * pi * ( fc * td + (BW / Tp) * (td .* t) ))
           ; % M x N

        % Correlation: sum over radar positions and time
        pixel = sum(sum(meas .* phase, 2));    % sum over rows then
           cols -> scalar
        img(ix, iz) = abs(pixel);
    end
end
t_focus = toc;
```

```
% ---------- 7) Range-squared compensation ----------
% z_range is 1 x N; we want to multiply each column (range bin) by z
   ^2:
img = img .* (ones(M,1) * (z_range .^ 2));   % M x N .* M x N

% ---------- 8) Display SAR Image ----------
figure;
imagesc(z_range, x_range, img ./ max(img(:)));
axis xy;
xlabel('Down-range z (m)');
ylabel('Cross-range x (m)');
title(sprintf('SAR TDC | Fs=%.0f kHz | fc=%.2f GHz | Rail length=%.1f
   m', Fs/1e3, fc/1e9, (max(xp)-min(xp))));
colormap parula; colorbar;

fprintf('Processing time: %.3f s\n', t_focus);
fprintf('Range resolution  R  = %.2f m, Max range Rmax = %.1f m (N=%d)
   \n', dR, Rmax, N);

% ---------- Cross Range Resolution ----------
L = max(xp) - min(xp);              % aperture length
R_ref = mean(z_pos);                % reference range (average target
   depth)
delta_x = lambda * R_ref / (2 * L);
fprintf('Cross Range Resolution = %.2f m\n', delta_x);
```

Listing 7: Code for Range Migration Algorithm for simulated targets

```
close all

% Radar parameters
Fs = 50e3;
Tp = 5e-3;
f_start = 2.4e9; f_stop = 2.5e9;


% Data generation inputs
c = 299792458;
x_pos = [-2050e-3 0e-3 2500e-3]; y_pos = [0 0 0]; z_pos = [10 20 30];
fc = (f_start + f_stop) / 2;
lambda = c/fc; spacing = lambda/2;
xp = -3000e-3:spacing:3000e-3; yp = 0;
N = floor(Tp*Fs);
t = linspace(0, Tp, N);
BW = f_stop - f_start;

% Generate data
SAR_data = SAR_Data_Generation(x_pos, y_pos, z_pos, xp, yp, N, t, fc,
   BW, Tp);
data=squeeze(SAR_data);

% Generate kx_2D
delta_kx = pi/max(xp); % 2*pi/(2*max(xp))???
kx = delta_kx * ( -(size(xp, 2)-1)/2 : (size(xp, 2)-1)/2 );
```

```
kx_2D = repmat(kx.', [1,N]);

% Generate kt_2D
kt = 2*pi*fc/c + (2*pi*BW/Tp)*(t/c);
kt_2D = repmat(kt, [size(xp,2),1]);

% Generate kz_2D
kz_2D = sqrt(4*(kt_2D.^2) - kx_2D.^2);

% Generate uniform kz
kz_1D_uni = linspace(min(min(kz_2D)), max(max(kz_2D)), size(kz_2D,2));
kz_2D_uni = repmat(kz_1D_uni,[size(xp,2),1]);

% Plot the K-space both the nonuniformly and uniformly sampled kz for
   comparison
scatter(kx_2D(1:10:end), kz_2D(1:10:end)); hold on
scatter(kx_2D(1:10:end), kz_2D_uni(1:10:end));
xlabel('k_x');
ylabel('k_z');

% Cross range 1-D FFT with no zero padding
S_B = fftshift(fft(SAR_data,[],1),1);

% Stolt interpolation
S_B2 = zeros(size(xp, 2), N);
for i = 1:size(xp, 2)
    S_B2(i,:) = interp1(kz_2D(i,:), S_B(i,:), kz_1D_uni);
end
S_B2(isnan(S_B2)) = 1e-30;

% 2-D IFFT
ft_1 = ifft(S_B2,[],1);
ft_2 = ifft(ft_1,[],2);

% Generate the axis for the final image
delta_x = pi/max(kx); % 2*pi/(2*max(kx))???
x_axis = delta_x * ( -(size(xp, 2)-1)/2 : (size(xp, 2)-1)/2 );
delta_fz = c * (max(kz_1D_uni) - min(kz_1D_uni)) / (4*pi);
Rmax = N*c / (4*delta_fz);
z_axis = linspace(0, Rmax, N/2);

% Multiply each column, element by element by square of the range axis
   .
% Use lower half of the data
f_corrected = ft_2(:,1:size(ft_2,2)/2).* z_axis.^2;

figure
imagesc(z_axis, x_axis,(abs(f_corrected/max(f_corrected(:)))));
colorbar;
title('SAR Image');
xlabel('Z (m)');
ylabel('X (m)');
```

```
function SAR_Data_Time_Domain = SAR_Data_Generation(x_pos, y_pos,
   z_pos, xp, yp, N, t, fc, BW, Tp)

    c = 299792458; %(m/s) speed of light
    f = 1*ones(1, length(x_pos)); % reflectivity function for the
       targets
    n_targets = length(x_pos); % Total number of targets
    Tot_x = length(xp); % Total number of x positions/measurements
    Tot_y = length(yp); % Total number of y positions/measurements

    SAR_Data_Time_Domain = zeros(Tot_x,Tot_y,N);

    for nt = 1:n_targets
        for m = 1:length(xp)
            r = sqrt((x_pos(nt)-xp(m)).^2 + y_pos(nt).^2 + z_pos(nt)
               .^2);
            tau = 2 * r / c;
            phase = cos((2*2*pi*fc*r/c)+(2*2*pi*BW*r*t/(c*Tp)));  % 1
                N
            SAR_Data_Time_Domain(m, 1, :) = SAR_Data_Time_Domain(m, 1,
                :) + reshape(f(nt)*(1./(r.^2)).*phase, 1, 1, []);
        end
    end
end
```

Listing 8: Code for Range Migration Algorithm for real targets

```
clearvars -except sync backscatter Fs
clc
close all

% Read file
[data, Fs] = audioread('SAR_25sep_edited.wav');
if size(data,2) == 1
    error('Input audio must have two channels: backscatter and sync (
       square).');
end


sync = data(:, 2);
backscatter = data(:, 1);
clear data
%
% figure;
%
% subplot(2,1,1)   % 2 rows, 1 column, first plot
% plot(backscatter);
% ylabel('Amplitude');
% xlabel('Data Sample Number');
% title('Radar backscatter data');
%
% subplot(2,1,2)   % 2 rows, 1 column, second plot
% plot(sync);
```

```
% ylabel('Amplitude');
% xlabel('Data Sample Number');
% title('Sync data');



%%
% Make sync binary using a small adaptive threshold
thr_find_pos = 0.05;
thr = 0;
sync_data = sync > thr_find_pos;


% Expected number of samples per up-chirp
Trp = 0.3;               % range profile time in seconds (e.g. 275 ms)
Nrp = round(Trp * Fs);  % samples per range profile
guard = Nrp;
Tp = 0.005;

% Detect sync pulse edges
d = diff([0; sync_data; 0]);
starts = find(d == 1); % When each chirp starts
ends   = find(d == -1) - 1; % When each chirp ends

% Remove chirps that are too short or too long
for i = 1:length(starts)
    chirp_length = ends(i) - starts(i);
    if chirp_length < 0.5*Tp*Fs || chirp_length > 2*Tp*Fs
        starts(i) = 0; ends(i) = 0;
    end
end
starts(starts == 0) = []; ends(ends == 0) = [];

% Remove chirps that are too close together
for i = 1:length(starts)-1
    spacing_length = starts(i+1) - ends(i);
    if spacing_length < 0.5*Tp*Fs || spacing_length > 2*Tp*Fs
        starts(i) = 0; ends(i) = 0;
    end
end
starts(starts == 0) = []; ends(ends == 0) = [];


chirp_times = ends - starts;
silence_after = [starts(2:end); length(sync_data)]-ends(1:end); % How
   much silence is after each chirp
silence_before = [inf; silence_after(1:end-1)]; % How much silence is
   before each chirp
long_silences = silence_before > Nrp;

pos_starts = starts .* long_silences;
pos_starts(pos_starts == 0) = []; % Start time of each chirp that is
```

```
   preceeded by a long silence

sync_data = sync > thr;

% test123 = zeros(1, length(sync));
% test123(pos_starts) = 2;
% len_over_10 = floor(length(sync)/10);
% for i = 0:9
%     idx = len_over_10*i+1:len_over_10*(i+1);
%     figure
%     plot(sync(idx)); hold on; plot(test123(idx));
% end
% plot(sync); hold on; plot(test123);
% plot(sync(round(end*0.3):round(end*0.5)));hold on; plot(test123(
   round(end*0.3):round(end*0.5)))
% plot(sync(1:pos_starts(1)+Nrp*2));hold on; plot(test123(1:pos_starts
   (1)+Nrp*2))
% test321 = zeros(size(test123));
% test321(pos_starts(1)+Nrp) = 2;
% plot(test321(1:pos_starts(1)+Nrp*2))

% test123 = zeros(1, length(sync));
% for i = 1:length(pos_starts)
%     pos = pos_starts(i);
%     test123(pos + Nrp : pos + 2*Nrp-1) = sync(pos + Nrp : pos + 2*
   Nrp-1);
% end
% plot(sync(1:2000000))
% hold on
% plot(test123(1:2000000))
% title('Sync data')
% ylabel('Amplitude');
% xlabel('Data Sample Number');
% legend('Sync data', 'Parsed sync data')


%%
% Number of positions (one per sync pulse)
M = round(length(pos_starts));

% Keep only the first tablesworth of measurments
% third = floor(length(pos_starts)/3);
% pos_starts = pos_starts(1:third);
% M = length(pos_starts);

% Keep only an odd number of radar positions
if mod(M, 2) == 0
    M = M - 1;  % discard the last position if even
    pos_starts(end) = [];
end


%%
```

```
% Allocate matrices
DataMatrix = zeros(M, Nrp);
SyncMatrix = zeros(M, Nrp);

for m = 1:M
    s = pos_starts(m) + guard;  % start index after guard band
    if s+Nrp-1 <= length(backscatter)
        DataMatrix(m,:) = backscatter(s:s+Nrp-1);
        SyncMatrix(m,:) = sync_data(s:s+Nrp-1);
    else
        warning( Position %d truncated (not enough samples). , m);
    end
end

Tp = 0.005;
N = floor(Tp*Fs/2)*2;

% 1 = DC, N/2+1 = Nyquist

ProcessedUpChirps = zeros(M, N);
%ProcessedUpChirps = zeros(round(M/2), N);

for m = 1:M
    data_row = DataMatrix(m, :);
    sync_row = SyncMatrix(m, :);

    % Count number of up-chirps by detecting edges
    % d_sync = diff([0 sync_row 0]);
    % upchirp_starts = find(d_sync == 1);
    % upchirp_ends   = find(d_sync == -1) - 1;
    % numUpChirps = length(upchirp_starts);

    d_sync = diff(sync_row);
    upchirp_starts = find(d_sync == 1) + 1;
    upchirp_ends   = find(d_sync == -1);

    if upchirp_ends(1) < upchirp_starts(1)
        upchirp_ends(1) = [];
    end
    if upchirp_ends(end) < upchirp_starts(end)
        upchirp_starts(end) = [];
    end
    if length(upchirp_starts) ~= length(upchirp_ends)
        disp('FAULTY CHIRPS!')
        disp(upchirp_starts)
        disp(upchirp_ends)
    end
    numUpChirps = length(upchirp_starts);

    if numUpChirps == 0
        warning( No up-chirps found at position %d , m);
        continue
    end
```

```
    % Sum each integrated upchirp
    upchirp_sum = zeros(1, N);
    for i = 1:numUpChirps
        chirp_data = data_row(upchirp_starts(i):upchirp_ends(i));
        if length(chirp_data) < 10
            disp(length(chirp_data));
        end
        % Make the chirp data exactly N long
        t = linspace(0,1,length(chirp_data));
        tN = linspace(0,1,N);
        chirp_data = interp1(t, chirp_data, tN);

        upchirp_sum = upchirp_sum + chirp_data;
    end

    % Average over number of up-chirps
    avg_upchirp = upchirp_sum / numUpChirps;

    % Apply Hann window
    w = hann(length(avg_upchirp))';
    avg_upchirp = avg_upchirp .* w;

    % Hilbert transform
    avg_upchirp = hilbert(avg_upchirp);
    avg_upchirp(isnan(avg_upchirp)) = 1e-30;

    % Put the values into the matrix
    ProcessedUpChirps(m, :) = avg_upchirp;
%     if m == 5   % <-- choose position you want to visualize
%     figure;
%
%     % Plot raw data with sync overlay
%     subplot(2,1,1);
%     plot(data_row); hold on;
%     yyaxis right
%     plot(sync_row, 'r'); % sync signal
%     yyaxis left
%     xlim([0 5000])
%
%     % Mark chirp start/ends
%     for i = 1:numUpChirps
%         xline(upchirp_starts(i), 'g--');
%         xline(upchirp_ends(i), 'r--');
%     end
%     title(sprintf('Data Row with Sync (Position %d)', m));
%     xlabel('Sample Index'); ylabel('Amplitude');
%     grid on;
%
%     % Plot averaged chirp
%     subplot(2,1,2);
%     plot(real(avg_upchirp), 'b', 'LineWidth', 2);
%     title(sprintf('Averaged Up-Chirp (Position %d)', m));
```

```
%     xlabel('Sample Index'); ylabel('Amplitude');
%     grid on;
% end

end


%%
% figure
% imagesc(ProcessedUpChirps);
% xlabel('Up-chirp fast time (samples)'); ylabel('Rail position');
% title('Integrated up-chirp data (rows=positions, cols=fast-time)');

%%
% Radar parameters
Tp = 5e-3;
f_start = 2.4e9; f_stop = 2.5e9;

% Fast time axis
t = linspace(0, Tp, N);

% Slow time / radar rail parameters
spacing = 0.06;           % step size (m)
L = spacing * size(ProcessedUpChirps,1);   % total rail length (m)

% Slow time axis / radar positions
xp = linspace(-L/2, L/2, size(ProcessedUpChirps,1));

% Constants
c = 299792458;               % Speed of light (m/s)
fc = (f_stop + f_start)/2;  % Carrier frequency (Hz), midpoint of
   2.4-2.5 GHz
BW  = f_stop - f_start; % Bandwidth (Hz)

% Generate kt
kt = 2*pi*fc/c + (2*pi*BW/Tp)*(t/c);

% figure
% imagesc(kt, xp, angle(ProcessedUpChirps));
% xlabel('K_t (rad/m)');
% ylabel('Synthetic aperture position x_p (m)');
% title('Phase before along track FFT');

% Desired zero-padding size
zpad = 2049;

% Calculate number of zeros to add on top and bottom
pad_top    = floor((zpad - M)/2);
pad_bottom = ceil((zpad - M)/2);

% Add zero-padding symmetrically
ProcessedUpChirps_zp = [zeros(pad_top, N); ProcessedUpChirps; zeros(
   pad_bottom, N)];
```

```
% Current size
M = size(ProcessedUpChirps_zp, 1); % M = 2049

% Updated L and xp
L = spacing * M  ;   % total rail length (m)
xp = linspace(-L/2, L/2, M);

% Generate kx_2D
delta_kx = pi/max(xp); % 2*pi/(2*max(xp))???
kx = delta_kx * ( -(size(xp, 2)-1)/2 : (size(xp, 2)-1)/2 );
kx_2D = repmat(kx.', [1,N]);

% Generate kt_2D
kt_2D = repmat(kt, [size(xp,2),1]);

%Generate kz_2D
kz_2D = sqrt(4*(kt_2D.^2) - kx_2D.^2);

% Generate uniform kz
kz_1D_uni = linspace(min(min(kz_2D)), max(max(kz_2D)), size(kz_2D,2));
kz_2D_uni = repmat(kz_1D_uni,[size(xp,2),1]);

% figure
% scatter(kx_2D(1:10:end), kz_2D(1:10:end));
% hold on
% scatter(kx_2D(1:10:end), kz_2D_uni(1:10:end));
% title('K_z comaprison');

% Perform 1-D FFT
S_B = fftshift(fft(ProcessedUpChirps_zp,[],1),1);

% figure
% imagesc(kt, kx, 20*log10(abs(S_B)), [-18, 20.5]);
% xlabel('K_t (rad/m)')
% ylabel('K_x (rad/m)')
% title('Magnitude after along tarck FFT');
% colorbar;
%
% figure
% imagesc(kt, kx, angle(S_B));
% xlabel('K_t (rad/m)')
% ylabel('K_x (rad/m)')
% title('Phase after along tarck FFT');
% colorbar;
%%

% Stolt interpolation
S_B2 = zeros(size(xp, 2), N);
for i = 1:size(xp, 2)
    S_B2(i,:) = interp1(kz_2D(i,:), S_B(i,:), kz_1D_uni);
end
S_B2(isnan(S_B2)) = 1e-30;
```

```
% figure
% imagesc(kz_1D_uni, kx, angle(S_B2))
% xlabel('K_z uniform (rad/m)')
% ylabel('K_x (rad/m)')
% title('Phase after along tarck FFT');
% colorbar;

% 2-D IFFT
ft_1 = ifft(S_B2,[],1);
ft_2 = ifft(ft_1,[],2);

% Create indices
delta_fz = c * (max(kz_1D_uni) - min(kz_1D_uni)) / (4*pi);
delta_kz = 4*pi*delta_fz/c;
Rmax = N*c/(2*delta_fz);
delta_x = 0.06; % ???
Rail_Rmax = size(ft_2, 1) * delta_x;

d_range_1 = 1; d_range_2 = floor(Rmax/4);
c_range_1 = -10; c_range_2 = 10;

d_index_1 = ceil((size(ft_2, 2)/Rmax)*d_range_1);
d_index_2 = ceil((size(ft_2, 2)/Rmax)*d_range_2);

c_index_1 = ceil((size(ft_2, 1)/Rail_Rmax)*(c_range_1+(Rail_Rmax/2)));
c_index_2 = ceil((size(ft_2, 1)/Rail_Rmax)*(c_range_2+(Rail_Rmax/2)));

% Flip and rotate
ft_2 = rot90(ft_2, 1);
ft_2 = fliplr(ft_2);

% Truncate
truncated_data_matrix = ft_2(d_index_1:d_index_2, c_index_1:c_index_2)
   ;

% Create downrange and crossrange
downrange = linspace(-1*d_range_1, - 1*d_range_2, size(
   truncated_data_matrix, 1));
crossrange = linspace(c_range_1, c_range_2, size(truncated_data_matrix
   , 2));

% Combine truncated data with downrange vector
truncated_data_matrix = (downrange(:).^2) .* truncated_data_matrix;

% Create image
image = 20*log10(abs(truncated_data_matrix));

% Plot image
figure
imagesc(crossrange, downrange, image, ...
    [max(max(image))-25, max(max(image))+0]);
colorbar;
```

```
xlabel('Crossrange (meter)');
ylabel('Downrange (meter)');
title('Final image');
```

Listing 9: Code for SDR range measurements

```
clear; clc; close all

%% Load data and set parameters
use_IQ = true;

use_LowIF = true;

if use_IQ
    [I_data, Fs] = audioread('SAR_data/breath_lowIF_I.wav');
    [Q_data, Fs] = audioread('SAR_data/breath_lowIF_Q.wav');
else
    [data, Fs] = audioread('SAR_data/single_lowIF_I.wav');
end

c  = 299792458; % Speed of light
f0 = 5.8e9; % Base radar frequency

if use_LowIF
    f0 = f0 + 4e3;
end

if use_IQ
    data = complex(I_data,Q_data);
end

% Remove first 3 seconds
data = data(3e6:end);
if use_IQ
    I_data = I_data(3e6:end);
    Q_data = Q_data(3e6:end);
end

%% Plot the recorded data
% t = (0:length(data)-1)/Fs;
% figure
% if use_IQ
%     subplot(2,1,1); plot(t, I_data); grid on;
%     xlabel('Time (s)'); ylabel('Amplitude'); title('Recorded I_data
   ')
%     subplot(2,1,2); plot(t, Q_data); grid on;
%     xlabel('Time (s)'); ylabel('Amplitude'); title('Recorded Q_data
   ')
% else
%     plot(t, data); grid on;
%     xlabel('Time (s)'); ylabel('Amplitude'); title('Recorded data')
% end

%% Data processing
```

```
if ~use_IQ
    data = hilbert(data);
end
phi = unwrap(angle(data));
R = (c/(4*pi*f0)) * phi;

%% Plot the
t = (0:length(data)-1)/Fs;
figure
plot(t, 1e3*R);
grid on;
xlabel('Time (s)'); ylabel('Range (mm)');
title('Range')
exportgraphics(gcf,'myplot.pdf','ContentType','vector');

%%
N = length(data);
F = fft(R, 1*N);
F = F(1:0.5*N);
f = linspace(0, Fs/2, 0.5*N);
figure
plot(f, 20*log10(abs(F)), 'LineWidth', 2)
xlim([0,3])
grid on;
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('FFT of range')
```